

Sheepdog, Parallel Collaborative Programming-by-Demonstration

Vittorio Castelli⁽¹⁾, Lawrence D. Bergman⁽²⁾, Tessa Lau⁽³⁾, and Daniel Oblinger⁽⁴⁾ *

February 4, 2007

Abstract

We introduce *parallel collaborative programming-by-demonstration* (PBD) as a paradigm for capturing knowledge on how to perform computer-based procedures by independently recording multiple experts executing these tasks and combining the recordings via a learning algorithm. Traditionally, PBD has focused on end-user programming, and does not support well parallel collaborative procedure model construction from examples. In this paper we discuss how to appropriately extend the main aspects of PBD (instrumentation, abstraction,

***Index Terms:** Procedural Knowledge, Programming by Demonstration, Hidden Markov Models.

IBM Thomas J. Watson Research Center P.O. Box 218, Yorktown Heights, NY 10598.

(1) e-mail: vittorio@us.ibm.com

(2) e-mail: bergmanl@us.ibm.com

(3) e-mail: tlau@us.ibm.com

(4) e-mail: daniel.oblinger@DARPA.mil

learning, and execution), and we describe the implementation of these extensions in a system called Sheepdog.

1 Introduction

Programming-by-demonstration (PBD) [1, 2] is a class of techniques typically used for improving end-user productivity by automating repetitive tasks or for constructing graphical applications. PBD is mostly considered as an end-user-programming tool, where a single author demonstrates the task.

In this paper, we propose a different perspective on the field by introducing the concept of *collaborative PBD*, in which multiple authors contribute demonstrations to an appropriate learning algorithm that is responsible for combining them and producing a distributable procedure model. We concentrate in particular on *parallel collaborative PBD*, in which multiple recordings of the same task are collected in parallel and combined simultaneously by the learning algorithm.

A motivating scenario for parallel collaborative PBD is offered by technical support for computer systems. In a typical call center, technical support personnel often solve customer problems by remotely controlling the customer’s computer. The same problem is solved over and over by different experts, who perform variations of the same procedure that differ depending on the configuration of the customer’s hardware and software or on the

root cause of the problem. PBD offers an appealing solution to reducing the cost of technical support: by recording the solutions provided by the different experts and combining them via an appropriate learning algorithm, it is possible to produce a model of the task that can be used to guide the end-user in a self-help scenario or to fully automate the solution.

The Sheepdog system described in this paper demonstrates the feasibility of using parallel collaborative PBD for technical support. Its design was in part driven by the results of an informal survey of documented technical support and maintenance procedures, as well as from discussions with technical support personnel. The following conclusions were instrumental in the design of Sheepdog:

1. Technical support procedures typically have a number of branches that reflect variations in the environment within which they are performed. For example, a procedure for establishing network connectivity has different subprocedures depending on whether the connection is dialup or broadband.
2. Documentation and/or scripts rarely cover the full set of conditions that are encountered by actual users. Unexpected errors or unanticipated configurations occur frequently.
3. Both documentation and scripts quickly become obsolete when the underlying system changes, and must be updated often to remain useful.

4. Many maintenance procedures require human intervention at certain points and thus are not amenable to “one-click” automation. This is particularly true when unexpected errors occur, and the user must formulate a recovery strategy, or when the consequences of taking a wrong action are very costly.

These considerations extend to other applications domains, where procedures range from transaction processing to systems administration, to application development tasks.

This paper makes the following contributions:

- Parallel collaborative PBD as a new paradigm for PBD;
- An architecture that extends the main PBD components (recording, learning, authoring, and execution) to support parallel collaborative PBD.
- A novel learning algorithm for PBD based on Input/Output Hidden Markov Models [3];
- An implemented system, Sheepdog, that learns technical support procedures by demonstration; and
- An empirical evaluation of Sheepdog’s learning algorithm based on data collected from a user study.

The rest of the paper is organized as follows. We begin with a simplified

usage scenario in Section 2, in which we outline the steps of recording, learning, authoring, and executing a Sheepdog procedure. In Section 3.3 we discuss the functions that an architecture supporting the scenario of Section 2 should provide, the corresponding implementation in the Sheepdog system and the limitations of the implementation. Section 4 contains the description of experimental results. Section 5 outlines related work. Finally, in Section 6 we conclude with a discussion of the Sheepdog system, a sketch of the architectural and algorithmic open problems, and a summary of our contributions.

2 A Usage Scenario

In this section we describe a simple scenario that illustrates how Sheepdog could be used for technical support. We consider a situation in which a network printer is updated with new features and users need to change their settings to take advantage of these features.

The first user, Alice, calls the help desk for assistance in configuring a printer. The desk-side support representative (DSR), Bob, obtains the name of Alice’s machine, and connects to it (e.g., with remote desktop). Bob turns on Sheepdog on Alice’s machine and presses the “record” button on the Sheepdog GUI. He then performs the installation. First, he navigates to the “Printers and Faxes” dialog, then opens the “Options” dialog for the desired printer, and finally checks whether the “Duplex” option is

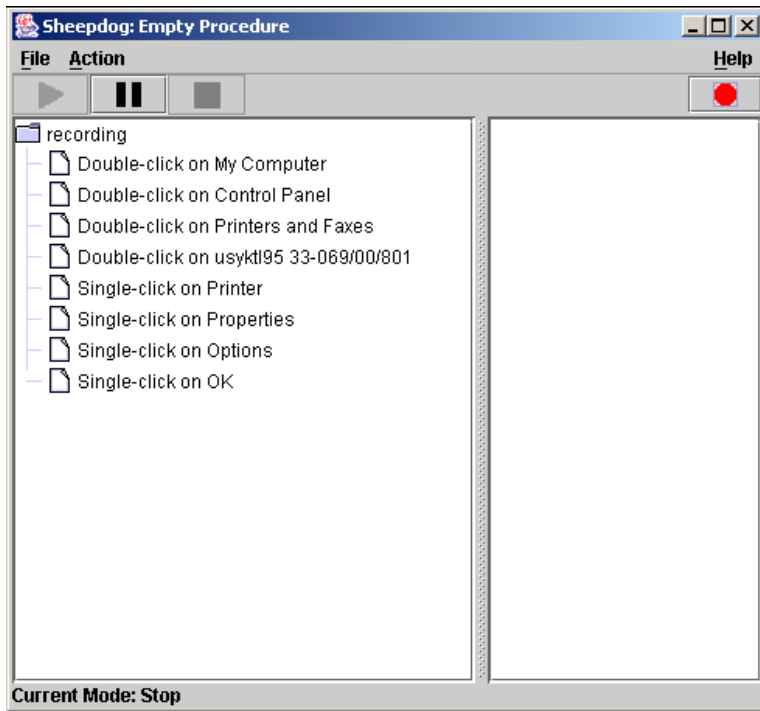


Figure 1: Initial recording of the procedure.

selected. In Alice's case, the option is already selected, and Bob clicks the OK button. While Bob is performing the procedure, Sheepdog records all the actions and shows them in the GUI (Figure 1). Finally, Bob closes all the open dialogs. The recording is then automatically sent back to the call center, and becomes part of the ticket. While Bob fills out the rest of the ticket, the recording is also turned into a procedure; the recording is then shown to Bob. Before closing the ticket, Bob lightly modifies (*authors*) the procedure representation produced by Sheepdog, to group steps into semantically meaningful groups. The result is shown in Figure 2. The procedure model is now ready to be distributed.

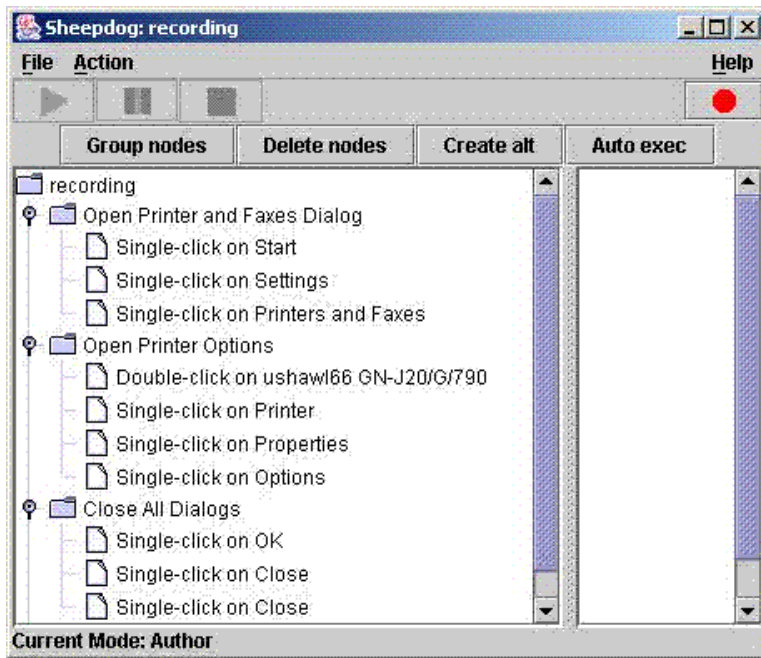


Figure 2: The procedure of the example, after editing the first recording.

The second user, Charlie, also calls the help desk for assistance. During the preliminaries of the phone call, the call tracking application prompts the DSR, David, of the existence of a Sheepdog procedure that could assist during the problem resolution. David selects the procedure, which is automatically loaded on Charlie's Sheepdog, remotely connects to Charlie's computer, and start configuring the printer with the assistance of Sheepdog. As shown in Figure 3, Sheepdog determines the first step to be performed and identifies the application and the widget involved in this step. Sheepdog changes the background of the first step in the procedure description (A in the figure) to yellow and highlights the corresponding widget on the

application UI (B in the figure). David has a choice: he can allow Sheepdog to perform the entire procedure, he can use the step-by-step automation facility, or he can perform the actions manually. David selects Sheepdog's single-step automation facility. When he reaches the configuration of the "Duplex" option, David observes that this option is not selected, takes over, and manually performs the step of selecting the appropriate option. Sheepdog observes and records these actions, compares them with the procedure model, detects that a deviation from the prescribed path has occurred, and refrains from suggesting the next step. As David subsequently performs a few steps that pertain to the loaded procedure model, Sheepdog matches them to the model and, after two subsequent matches, start offering suggestions on the next action to perform. At the end of the call, the new recording is automatically incorporated into the Sheepdog procedure by the learning algorithm. David performs additional editing and adds a decision point around the steps circled in Figure 4.

In general, forcing the DSRs to edit the Sheepdog procedure after performing previously unseen actions is not cost-effective. An alternative strategy is to wait until a predefined number of such recordings is observed before rebuilding the procedure and asking a DSR to perform the edits. Additionally, this task could be delegated to a higher-level DSR and performed when the call volume is low. The Sheepdog procedure can also be added to a self-help web site, to allow customers to solve the problem without calling

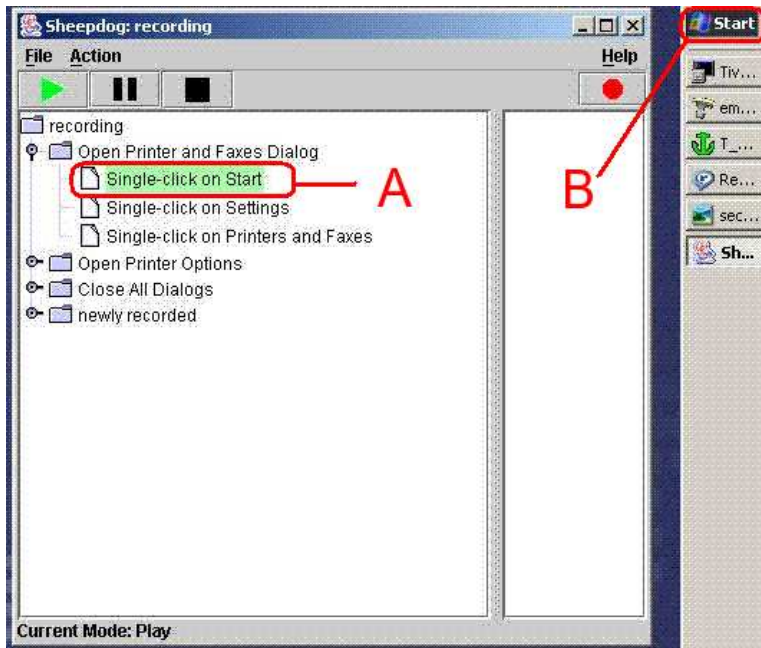


Figure 3: The Sheepdog GUI at the beginning of playback. Note that the first step of the procedure is highlighted (A), and that the corresponding widget, the “Start” button of the Windows task bar, is also highlighted by changing both foreground and background color (B).

the help center.

The described scenario is fictitious; however, most of the described infrastructure is common across support centers: the existence of a problem database that contains solutions or suggestions, tools for remotely controlling the desktops of customer’s machines, and communication mechanisms for uploading and downloading data from and to a customer’s computer are established and widely used technologies. This paper addresses the sci-

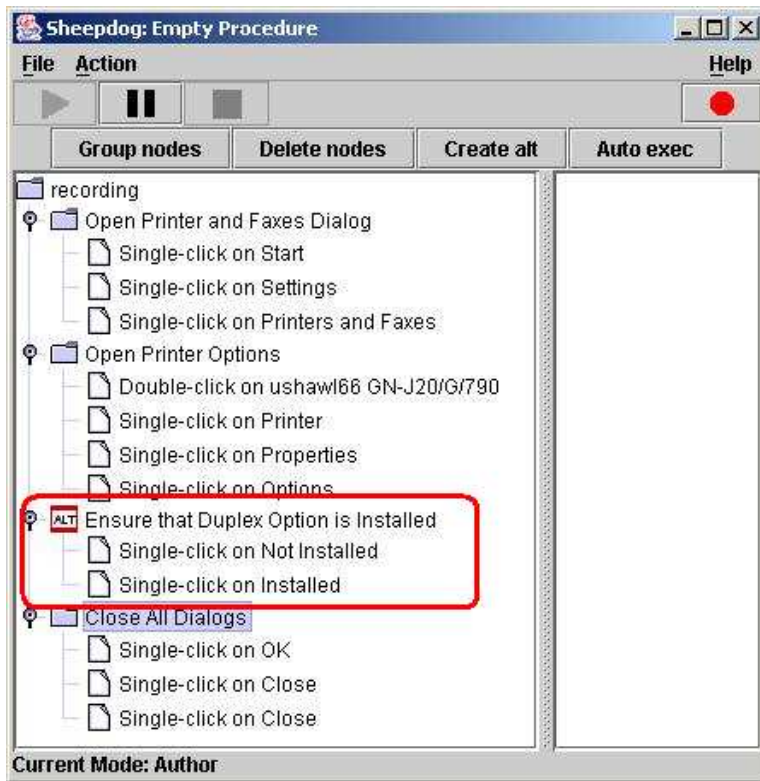


Figure 4: The procedure authored after the second recording, with the new decision point circled. Note that the decision point is marked by an icon labeled “ALT”. If the duplex is not selected, the user first clicks on the selection combo box, which has value equal to “Not Installed”, and then selects the “Installed” entry in the pull down menu that appears.

entific and technical problems associated with the missing piece: a system that learns procedure models from multiple demonstrations and produces artifacts that can be used to automate a task or to guide the user during its execution.

3 Supporting Programming By Demonstration

3.1 Events and actions

User interactions with an application generate *low level events* at the OS level, such as key presses and releases, mouse movement, mouse down and up clicks, and mouse double-clicks. When a low level event is detected, the OS sequentially notifies all the *callback functions* registered for that event. The registered callback functions include, but are not limited to, those of the application owning the control activated by the user. When the application updates its GUI, for example, in response to a user interaction, other events are generated at the OS level, to indicate that changes to the GUI have occurred. We call these events *reactions* to the user actions. Reactions include creation of new windows, deletion of existing windows, change in the content of windows, etc. Reactions and low level events have the same notification and registration mechanism.

The term *user action* denotes in this paper an individual interaction of the user with an application as described in a step-by-step user guide. A user action typically corresponds to multiple low-level events and application reactions: for example, a drag-and-drop action corresponds to a sequence of mouse clicks and mouse moves, and of responses denoting the changes in the source and destination windows.

3.2 The Snapshot-Action Model of Interaction

This paper relies on a common model of user/application interaction, the *snapshot-action* model. Sheepdog represents the interaction between user and application using a conversational turn-taking paradigm. At the beginning of each turn, the system resides in a particular *system state*, part of which is reflected on the content of the GUI (*GUI state*); using the content of the GUI and information gathered from previous interaction, the user chooses an action to perform, such as clicking a button or selecting a menu item. In the next turn, the system updates its internal state in response to the user’s action and reflects the change in the user interface. The interaction is complete when all changes are reflected in the user interface. Each interaction can be captured by an snapshot-action pair, where the *snapshot* is a description of the state of the GUI prior to the user operation, and the *action* is a description of the user action performed by the user (the context will clarify whether the term ‘action’ corresponds to a description or to actual user operations).

The snapshot-action model is motivated by the assumption that the user performs an action based on a mental model of the procedure, on the current location within the procedure, and on the content of the screen. The application receiving the action executes it, updates its internal (invisible) state, and provides feedback to the user by changing the content of the screen.

Using a control theory analogy, the content of the screen is the observable state of the application and, if the UI is well designed, exposes all the information that the user needs to decide which action to perform next. The snapshot-action model is somewhat similar to the STRIPS representation used in planning [4], where the snapshot captures facts that are true in the world, and plan operators describe the possible means for altering the state of the world.

3.3 The Components of a PBD system

We now analyze the functionalities required to support PBD (summarized in Figure 5): instrumentation, abstraction, learning, and execution.

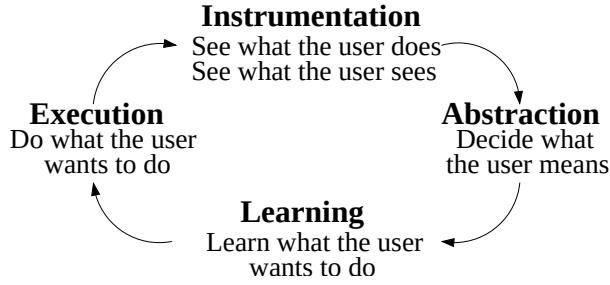


Figure 5: Essential functionalities of a general architecture for learning procedural knowledge by demonstration.

Instrumentation. To collect the demonstration data and to support execution, a PBD system must capture the user actions, the state of the computer system (in particular, of the GUI), and the reactions of the

OS and of the applications to the user actions. The *instrumentation* component is responsible for providing these forms of information capture. Section 3.4 contains a discussion of instrumentation in further details, followed by a description of the instrumentation layer of the Sheepdog system.

Abstraction. Learning a procedure in terms of the low-level events produced by the instrumentation can be extremely difficult; learning is much easier if the interaction between user and application is represented at a higher level of abstraction. Consider the task of launching a web browser. Different users perform this task in different ways, depending on individual preferences and configuration. Clearly, it is easier to learn the single step “Launch Web Browser” than the alternative sequences of key presses and mouse clicks performed by different users to accomplish this very same task. We use the term *abstraction* to denote the process of interpreting the stream of information captured by the instrumentation to produce a sequence of high-level steps. We further explore the topic of abstraction in Section 3.5, where we also describe the abstraction infrastructure of the Sheepdog system.

Learning. To support our motivating scenario, a PBD system must be able to take as input a collection of abstracted recordings of how different users executed the procedure (we call these *traces*) and combine them to produce an executable procedure model. We use the term *learning*

to denote this process. Section 3.6 is devoted to a general discussion of learning procedural knowledge, and to the learning components of the Sheepdog system.

Execution. The execution component assists a user in performing a task captured by a procedure model. Assistance can vary from full automation, where the execution component executes the procedure on the target application, to step-by-step guidance, where the execution component determines the next action to be performed and provides visual guidance to the user, who is responsible for performing the interactions with the application.

In this paper we both discuss the problems that arise with each of the four main PBD components in parallel collaborative PBD and describe how Sheepdog addresses them. We will base our discussion on the conceptual architecture shown in Figure 6. Here, an instrumentation layer interfaces with the Windows[®] platform by registering callback functions for specific events, and constructs a representation of the content of the screen by appropriately querying each window’s content using messages or appropriate APIs. An abstraction layer analyzes the information produced by the instrumentation and yields a snapshot-action model of the user/system interaction, where each user action is paired to a selected representation of the content of the screen just prior to the action itself. Sequences of state-action pairs are combined into a procedure model that can then be executed. During execution,

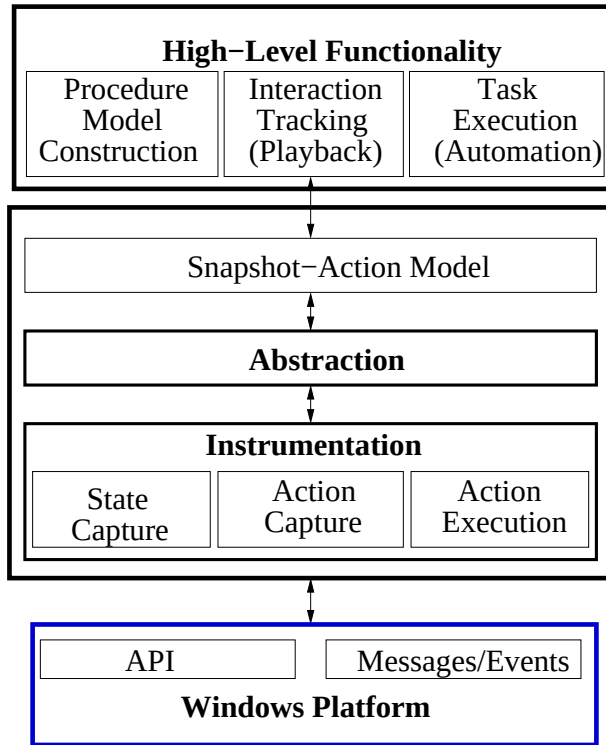


Figure 6: Conceptual architectural model of the Sheepdog system.

the Sheepdog system provides automation functionality to the user, by using the procedure model to predict the next action and executing the action via an appropriate instrumentation module. Additionally, the interaction tracking module can be used to follow the user and manage deviations from the learned task module.

3.4 Instrumentation

3.4.1 Instrumenting an application or a platform

The goal of the instrumentation layer is to monitor the interaction between user and system, and describe it to the abstraction, learning, and execution layers. In this section, we discuss two functions: “seeing what the user sees” (performed by the data-capture instrumentation), and “seeing what the user does” (performed by the action-capture instrumentation).

The Data-Capture Instrumentation

The first purpose of the instrumentation is to construct the snapshots of the snapshot-action model, namely, descriptions of the information the user knows about the state of the applications involved in the task. We distinguish two types of information:

Information available to the user but typically not visible on the screen.

This class of data includes system configuration information (e.g., the name of the computer, the types of devices installed, whether the system is connected to a network, the type of connection, the type of operating system) and session information (e.g., the username, system parameters that can be customized by individual users, such as the semantics of single clicks and double clicks on desktop icons and whether the mouse is configured for a left-handed or a right-handed

user). Knowing system and configuration information allows a collaborative PBD system both to abstract the user actions (e.g., the PBD system needs to know how the mouse is configured to determine the semantics of a mouse left click) and to specialize the procedure model for specific users (e.g., by suggesting to a left-handed user to left-click a GUI widget in order to open a popup menu).

Information available to the user and visible on the screen. The instrumentation must produce a description of what the user sees on the screen. This description is used by the abstraction and learning layers to infer the procedure model, and by the execution layer to identify the widget to be highlighted or activated. To this end, the instrumentation layer must produce a description of the window hierarchy for each active application, and must detect which window is in the foreground (since typically the user is looking at the foreground window). For each window, the instrumentation must obtain a description of the layout and of the visible content. For example, the instrumentation must produce a description of the content of list view windows which includes the geometry and properties of the individual list items.

Desirable properties of the data-capture part of the instrumentation layer include:

Support Wide Widget-Type Coverage. The snapshot should contain information on as many different widget types as possible. To support system-wide, general procedure model construction, Sheepdog assigns the same importance to all types of widgets during the task model induction.

Produce Accurate Richness of Detail. A rich description of the properties of each widget is highly desirable for the task-model-induction algorithm, which uses snapshot information to explain differences in user behavior, and therefore needs to accurately know the properties of the widgets that can influence the user decisions. Also providing rich details is a necessary step to solving the problem of correctly identifying widgets (see Section 3.8.3); for this purpose, structural information about the widget hierarchy is also often required, and should be acquired by the instrumentation.

Provide Complete Screen Coverage. The snapshot representation should broadly cover the content of the application UI. In particular, all widgets that are visible (i.e., not occluded or minimized) can be both targets of execution and sources of information for automatic task-model induction. Although not visible, occluded widgets could be targets of execution.

The Action-Capture Instrumentation

The second purpose of the instrumentation is to capture the actions of the user. To this end, the instrumentation must be able to detect mouse clicks and key presses, as well as events generated by widgets in response to user interactions. Additionally, the instrumentation must provide information on the functionality of the widgets with which the user interacts (for example, widgets of class “Button” are often used to implement press buttons, check boxes, and radio buttons, and the abstraction layer needs to know the specific functionality of a button to translate a mouse click into a semantically meaningful action such as “select radio button”). j

Mouse Clicks. The instrumentation must recognize the button pressed (left, right, middle), the action (mouse down, mouse up), the position of the mouse where the clicks happen, and the specific widget or window component where the up and down clicks occur.

The instrumentation must also report the states of modifier keys (<SHIFT>, <ALT>, <CTRL>, etc.) when mouse events are detected, because they affect the semantics of mouse-based operations.

Key Presses. The instrumentation must detect user key presses, and report them together with the state of the modifier keys. The identity of the GUI control receiving the key press operation must also be reported by the instrumentation, as well as the position of the cursor (if

applicable) when the user action occurs. Modern operating systems often support keyboard shortcuts, that is, the mapping of sequences of key presses, usually involving modifier keys, to complex operations. Often, keyboard shortcuts are mapped to specific operations, where the mapping can be customized by the user. In Windows the mapping between keyboard shortcuts and actions can be retrieved from the operating system via an appropriate system call. The instrumentation should therefore provide means for retrieving such mapping, to allow the abstraction layer to produce descriptions of user actions that are independent of the specifics of how the user performed the action.

Widget Functionality. The semantics of a sequence of low-level operations depend on the type of window or widget in which they are performed. For example, a double click in an icon on the desktop can launch an application or open a window; the same double click on the title bar of a window can result in a window-resize action. The instrumentation layer must provide information needed by the abstraction layer to properly interpret the user actions. This information includes the type of the window/widget receiving the action (e.g., their window class), as well as the values of parameters used in creating the window (e.g., the “window style” in the Microsoft Windows operating systems).

Finally, the instrumentation must detect the reaction of the system to user actions. In particular, the instrumentation must report the following

events:

Window creation and deletion, to allow the abstraction layer to maintain a coherent representation of which windows are in existence.

Changes in window geometry, namely, the location and the size of windows, required by the execution layer to perform selected operations.

Changes in window content and state, useful to the abstraction layer to interpret user actions, and to the learning algorithm to select the appropriate path when a decision point is encountered.

Changes in keyboard focus (namely, the window that receives the actions from the keyboard), and changes in *foreground window* (typically the window containing the widget with the keyboard focus), required by the abstraction layer to decide where the user is typing, and, potentially, by the execution layer (to prevent the unnecessary generation of SET_FOCUS events during the automatic execution of the procedure).

3.4.2 The Instrumentation Layer of the Sheepdog System

The instrumentation layer of the Sheepdog system relies on a variety of mechanisms to obtain the desired information from the Microsoft[®] Windows[®] 2000 and XP operating system.

System properties and system configuration parameters are retrieved from the Windows Registry. The username of the current user, the name of

the computer, the IP address of the different connections, and similar information, are all available from the Registry. In some cases, this information is more readily available through the Common Information Model (CIM) via the Windows Management Instrumentation (WMI); the Sheepdog system selects the best mechanism in a case-specific fashion.

To obtain a description of what the user sees on the screen, the Sheepdog system relies on appropriate calls to the Windows windowing system and to the Microsoft Active Accessibility component. The instrumentation layer retrieves the hierarchy of the windows displayed to the user. Retrieved properties of windows include the class name (used to infer its functionality); visual characteristics, such as dimension and styles; generic content and widget-specific content. To minimize the number of such calls, the instrumentation layer maintains a model of the content of the screen. This model is initialized by “snapshotting” the entire screen when the Sheepdog system is started, and incrementally updated when specific events are observed, such as user actions, creation and deletion of windows, changes of focus, and when the foreground window changes.

The instrumentation layer captures user actions and system reactions by means of “hooks” in the operating systems, that is, by registering system-wide callback functions for specific events, such as mouse clicks and window creations and deletion. The Windows 2000 and XP operating systems support a variety of hooks. The Sheepdog instrumentation uses five different

system hooks¹ to detect user and system events. When the instrumentation layer detects a user or a system event it selectively updates its model of the content of the screen, by making the appropriate system calls. More specifically, when an event is detected, the instrumentation layer identifies the properties of the display content that could have changed as a result of the event, retrieves only these properties (to minimize the number of system calls), and updates the internal model of the screen.

The Sheepdog instrumentation is general-purpose and is sufficient for a wide variety of applications. Nevertheless, it is not perfect, and suffers from three main limitations.

1. *Complexity*, due to the lack of a uniform way of obtaining all the necessary information from the operating system. As a consequence, the instrumentation generates a substantial number of system calls and this results in a visible impact on the performance of old, slow machines (this impact is much smaller on fast machines, especially with threaded, multiple-core, or multiple processors).

2. *Difficulty of operating in real-time within a concurrent environment.*

¹In particular, the following hooks are hooked: the Accessibility event hook, which detects a variety of windows events; the low-level keyboard hook, which detect key clicks; the low-level mouse hook, which detects and mouse events; the system message hook, which detects messages generated by events in dialog boxes, message boxes, menus, and scroll bars; and the Computer-Based-Training hook, that detects events such as window creation, deletions, change of focus, and change of foreground window.

For example, a request for the title of a window is translated by the OS into a message to the thread owning the window. This message does not have special priority and is served in a FIFO fashion; there is no guarantee that the response to the message contains the title of the window at the time the message was generated. In some instances, instances of selected window classes are allowed to refuse to respond to a message requesting information on their content if the request occurs when these windows are performing specific operations. Most of the complexity of the instrumentation results from dealing with a wide array of special cases.

3. *Lack of complete coverage.* This limitation stems from the fact that applications are not required to expose the content of their client area. The client area is typically visible only in those applications that implement it using standard window classes or subclasses of standard window classes (e.g., the Microsoft Notepad), or that expose the content for users with disabilities, using accessibility (e.g., Microsoft Internet Explorer).

3.5 Abstraction

3.5.1 Abstracting the information produced by the instrumentation

The Purpose of the abstraction layer is to extract semantic information from low-level events reported by the instrumentation. Describing the interaction between user and the applications at a higher semantic level to the learning algorithm has two benefits: 1. the learning task is facilitated, and 2. the learned procedure model can be concisely expressed in terms of high-level concepts. We analyze each benefit separately.

1. *Facilitating the learning task.* A high-level representation of the user-application interaction has several benefits for the learning algorithm, including:
 - It substantially reduces the number of steps required to represent a procedure execution. This allows the learning algorithm to induce a smaller, simpler procedure model.
 - It relieves the learning algorithm from the burden of accounting for the differences in the configuration of the individual computer systems. For example, the desktop of Microsoft Windows operating systems can be configured to launch applications with either single clicks or double click on desktop icons. The abstraction layer can present the learning algorithm with the same action in

both cases.

- It decouples the task of learning from that of filtering idiosyncrasies of user behavior. A task that a user performs entirely with the mouse can be performed by a different user entirely through the keyboard. The abstraction layer can reduce this source of variability by representing the user interaction in terms of the semantics of the actions rather than their actual execution.
- It reduces the effects of noise. A typical source of noise in traces is the presence of spurious actions, such as moving, minimizing, and resizing windows; pressing the wrong keys (and correcting the mistake using the backspace keys); and interacting with the wrong widget. Removing noise from traces simplifies the learning task because noisy actions cannot typically be explained by the learning algorithm.

2. *Producing a procedure description in terms of high-level concepts.* It is difficult for a user to understand procedure description in terms of the elementary user events reported by the instrumentation. It is therefore desirable for the abstraction layer to produce human-understandable high-level actions, such as **launch the web-browser**.

We distinguish the following abstraction levels, which correspond to different ways in which a human would describe a procedure:

Low-level Event, such as press the ‘a’ key, or release the left mouse button.

These events are produced by the instrumentation. A human would describe a procedure at this level of abstraction to a novice computer user, for example, to a small child.

User Action, such as: type the string ‘administrator’ in the type-in box labeled ‘user name’. This is the level of abstraction at which detailed instructions are written in how-to documentation, self-help instructions, and tutorials.

Execution Step, such as log on or open the ‘settings’ dialog of an application.

Note that each execution step typically corresponds to multiple user actions: for example, the log on step consists of opening the logon dialog, typing a username, and typing a password. We can distinguish *actional* execution steps, which have the effect of changing the state of the application on which they are performed, and *navigational* execution steps, which have the effect of making visible the part of the application where actional steps are executed. A human would use this level of abstraction to describe a task to an expert.

Subprocedure, such as “configure DNS settings”, as part of a procedure to install a new network card. This is the level at which a procedure is described in terms of subgoals. A human would provide a high-level overview of a task in terms of sub-procedures.

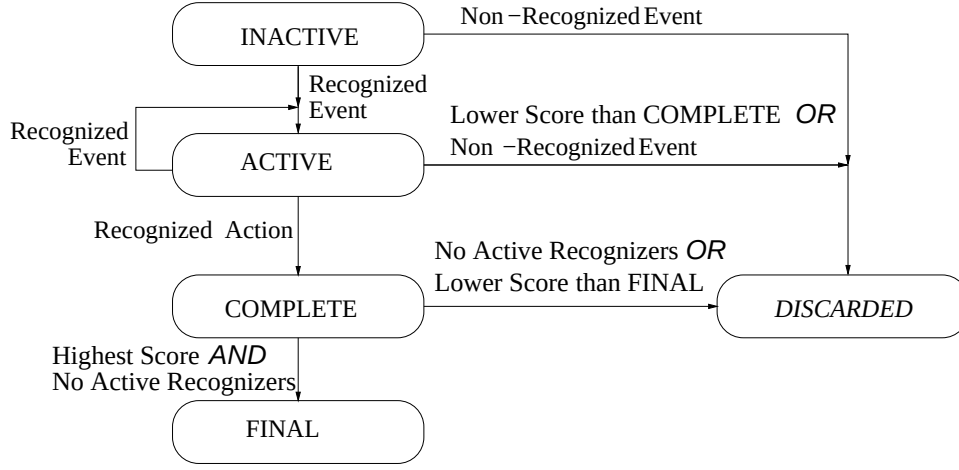


Figure 7: State-transition diagrams of recognizers.

Ideally, the abstraction layer and the induction algorithm should yield a procedure representation at multiple levels of abstraction, in order to provide the level of details appropriate to users of different skill and knowledge levels. We are not aware of any PBD system that support this feature.

3.5.2 The Abstraction Layer of the Sheepdog System

To support the scenario described in Section 2, Sheepdog must operate at a platform-wide, cross-application level. Consequently, its abstraction layer does not use application-specific knowledge but rather combines the low-level events detected by the instrumentation layer to produce a representation at the user action level. To provide support for higher-level semantics, Sheepdog offers an authoring facility, described in the next section, which allows the expert to specify hierarchical composition of sequences of user actions

into groups (*blocks*) that represent execution steps and subprocedures.

In general, mapping sequences of low-level events into user actions is a complex task. First, the same user action can be performed in different ways, for example, using menus rather than keyboard shortcuts. Second, different user actions can start with identical sequences of low-level events, or even consist of the same low-level events: for example, a mouse press, a mouse move, and a mouse release are the low level events associated with a drag-and-drop operation, with moving a slider or a scrollbar, and with resizing a window. It is also possible that the entire sequence of low-level events producing a specific action corresponds to the beginning of a different action. Consider for example the down click of the left mouse button: this low-level event could be the precursor of a single-click, a double-click, a click-and-drag user action, etc.. Therefore, in general it is not possible to recognize an action until it is completed, because, at each point in time, there can be multiple valid interpretations of a sequence of low-level events.

These and other considerations were fundamental to the design of the Sheepdog system abstraction layer. This layer consists of a collection of platform-dependent *recognizers*, finite-state automata that operate in parallel—each automaton recognizes a specific action—and a mechanism for disambiguating the actions produced by the recognizers, called the *segmentation controller*.

The operations of the recognizers (Figure 7 illustrates their state-transition

diagram) proceed as follows. Upon receiving a low-level event ϵ an **INACTIVE** recognizer transitions to the **ACTIVE** state if it recognizes the event as the first of the corresponding action; an **ACTIVE** recognizer transitions to the **COMPLETE** state if the sequence of events that ends with ϵ is recognized as a complete action; it remains in the **ACTIVE** state if ϵ is a valid event on the path to recognizing an action; and it becomes *DISCARDED* otherwise. Recognizers can also be in a *FINAL* state, used by the segmentation controller and described below.

Recognizers have an associated “abstraction” score used by the segmentation controller: recognizers of higher-semantic-level concepts have higher score than lower-semantic-level recognizers. For example, the double-click recognizer has a higher score than the single-click recognizer.

The operations of the segmentation controller proceed as follows. The segmentation controller maintains a current recognizer, $\mathcal{C}$, and a set of **ACTIVE** recognizers, $\mathbb{A}$. These are updated every time a low-level event is received. When the segmentation controller receives the first low-level event from the instrumentation, it instantiates a complete set of recognizers and passes the event to each of them. Recognizers that do not become **ACTIVE** or **COMPLETE**, become *DISCARDED* and are no longer invoked when the next low-level event is observed. Among the **COMPLETE** recognizers, the one with highest score is retained and becomes $\mathcal{C}$, the remaining ones become *DISCARDED*. Similarly, all the **ACTIVE** recognizers with score lower than that

of $\mathcal{C}$ are discarded, while those with a score equal or higher to that of $\mathcal{C}$ form the initial set $\mathbb{A}$. If $\mathbb{A}$ is empty, the **COMPLETE** recognizer becomes **FINAL**, the corresponding action is recognized, and the process is restarted.

In general, $\mathbb{A}$ is not empty, which means that the system cannot tell whether (1) the user performed the action recognized by $\mathcal{C}$ or (2) is in the process of performing one of the actions of the **ACTIVE** recognizers. Under the first hypothesis, the next low-level event is part of a new user action, under the second it is part of an action being performed and not yet completed. In order to simultaneously consider multiple hypotheses, the segmentation controller recursively maintains a chain of pairs $(\mathcal{C}^1, \mathbb{A}^1), (\mathcal{C}^2, \mathbb{A}^2), \dots$, called *successors* of $\mathcal{C}$, constructed as described above. When a new low-level event is observed, it is passed to each recognizer in $\mathbb{A}$, and recursively propagated through the chain of successors of $\mathcal{C}$. Note that the size of $\mathbb{A}$ is non-increasing: no new recognizers are added to it, while recognizers in $\mathbb{A}$ can become **DISCARDED** if they fail to recognize an incoming low-level event, or **COMPLETE** if they recognize an action. If one or more recognizers in $\mathbb{A}$ become **COMPLETE**, the one with highest score replaces $\mathcal{C}$, and all the successors of $\mathcal{C}$ are discarded. When $\mathbb{A}$ is finally empty, $\mathcal{C}$ becomes **FINAL**, and the corresponding action is recognized. The segmentation controller then replaces $\mathcal{C}$ and $\mathbb{A}$ with the immediate successors of $\mathcal{C}$, $(\mathcal{C}^1$ and $\mathbb{A}^1)$ and continues operating as described.

3.6 Learning

3.6.1 Inducing a procedure model

The purpose of the learning component is to convert a collection of abstracted recordings of procedure executions into an executable procedure model, that is, a model that can predict the correct actions during a future playback of the procedure. To support parallel collaborative PBD, the learning algorithm is faced with two main requirements not satisfied by the algorithms currently used in PBD.

Ability to learn in batch mode. Despite being a very appealing paradigm, incremental learning does not support the scenario of Section 2 where multiple DSRs could be simultaneously executing and refining the same procedure, or where the model is updated periodically, for example, when a prescribed number of new recordings has been collected. Therefore, we need a learning algorithm that operates in a batch mode

Ability to construct arbitrarily complex procedure structures from demonstrations alone. The learning algorithm must capture complex procedure structures that include decision points and iterations, and must accomplish this goal without requiring additional input from the author: the DSRs of our example cannot be distracted while performing their task by a PBD system that asks for help in inferring the procedure model.

Traditionally, two classes of learning algorithms have been used in PBD. Algorithms of the first kind support a single demonstration, and are typically used to construct models of repetitive tasks (i.e., a fixed sequence of operations on objects satisfying specific characteristics identified by the learning algorithm). Algorithms of the second kind can incrementally learn from multiple demonstrations. In some cases incremental learning consists of creating a collection of hypotheses with the first demonstration, and using the subsequent demonstrations to prune the hypothesis space and select one of the surviving models. When no additional user input is required, systems that learn from multiple demonstrations produce models with a simple structure, such as repetitive sequences of actions. Other systems can learn more complex procedure structures, for example containing decision points, by relying on additional user input beyond demonstrations, as described in the related work section. No system predating Sheepdog can automatically learn complex procedure structures from multiple demonstrations. Consequently, we developed a new class of PBD learning algorithms to support parallel collaborative PBD.

In this section we formally explore the implications of these two novel requirements and how they translate to a novel learning problem. We define the characteristics of a solution to this problem, and conclude the section with additional desirable characteristics of a learning algorithm for parallel collaborative PBD.

A learning algorithm induces a procedure model from a collection of *procedure demonstrations*, called a *training set* and denoted by $\mathbf{T}$. A procedure demonstration, or *trace* $\mathbf{t}$, is a sequence of *snapshot-action pairs* (SAPs) $(\mathbf{u}_1, \mathbf{y}_1), \dots, (\mathbf{u}_n, \mathbf{y}_n)$ observed while a user performs the task. A trace can be the recording of a complete or of a partial demonstration of the task. A *subtrace* is any contiguous subsequence of a trace. A *snapshot* $\mathbf{u}$ is a representation of the content of the GUI just prior to the execution of the action. The content of the GUI is summarized by extracting from each widget the values of the attributes that can change as a result of interactions between the user and the application. A snapshot can also contain information otherwise known to the user, such as the user name. An *action* $\mathbf{y}$ is a representation of a user's interaction with one or more applications, and must be specified to a detail sufficient to support automated execution. In this paper, the term action specifically refers to the output of the abstraction layer.

We represent a procedure model as a decorated graph, that is, a triple $(\mathcal{S}, \mathcal{E}, \mathcal{T})$. A (*procedure*) *step* $s \in \mathcal{S}$ is a pair (n, α) , where n is a node in the graph and the *action probability* α is a conditional probability measure on a finite set $\mathfrak{S}$ (defined below) of action skeletons given a snapshot $\mathbf{u}$. An *action skeleton* $\mathfrak{s}$ is a representation of a user action which can contain parameters (i.e., user-specified arguments, e.g., *select a file in the file-open dialog*) as well as variables (i.e., arguments that are evaluated at run time;

e.g., select the first file in the list, where the variable first file can be resolved to an actual value only while executing the procedure). We define $\mathfrak{S}$ to be the set of all the action skeletons in the procedure. A *directed edge* $e \in \mathcal{E}$ represents sequential ordering of two steps. Each edge in $\mathcal{E}$ has an associated *transition probability* $\tau \in \mathcal{T}$, that is, the conditional probability that the edge is followed given an observed snapshot. The transition probabilities associated with the collection of outgoing edges from a node are constrained to sum to 1 for every value of the snapshot. Thus, they collectively form a conditional probability measure on the next node given the snapshot and the current node.

Deterministic procedure models are special cases characterized by two defining properties: (1) each step is associated with a single action skeleton and (2) the allowed values of the transition probabilities are zero and one. More specifically, α puts all the probability on a single action skeleton irrespective of the snapshot, and the transition probability of one of the outgoing edges is equal to one while the others are equal to zero, for every value of the snapshot.

The term *procedure structure* denotes a directed graph $(\mathcal{N}, \mathcal{E})$ where each node $n \in \mathcal{N}$ is associated with a step, and the edges $\mathcal{E}$ do not have associated probabilities. We say that a procedure model is an *instantiated* version of its procedure structure.

Inducing a procedure model involves identifying a procedure structure, constructing a step for each node, and inferring the transition probabilities associated with the edges.

Snapshots are used during model construction for the induction of the action skeleton distributions and the transition probabilities. During playback, snapshots provide the context used for evaluating the transition probabilities and for instantiating the action skeletons it into a executable actions. For example, the action skeleton `select the first item in the 'Package Explorer' list` is instantiated to `select project1` when the state information shows that the first item in the list is called 'project1'.

Before proceeding, we address the important question of whether the learning algorithm should produce deterministic or probabilistic procedure models. For traditional PBD, deterministic models are generally sufficient. For parallel collaborative PBD, where the learning algorithm simultaneously combines a collection of independently recorded traces, deterministic models are no longer sufficient for two main reasons: different experts perform the same task slightly differently and even experts occasionally make mistakes. A deterministic model would try to explain all small variations and all errors. A probabilistic model, on the other hand, would assign higher probabilities to the most common pathways through the procedure (the *well-worn paths*), thus relegating idiosyncratic variants of the execution to a secondary role. Similarly, errors would be given smaller weight than correct paths.

Based on these considerations, the rest of the section will address probabilistic models. We first define the concept of alignment of a subtrace with a sequence of nodes in a procedure model as a labeling of the SAPs using steps as labels. We then define two learning problems in PBD, the simpler generalization problem, and the more complex alignment-and-generalization problem, and discuss the characteristics of their solutions (SimIOHMM, the learning algorithm of Sheepdog, is the first solution to the alignment-and-generalization problem). Finally, we discuss additional characteristics of learning algorithms that would be useful for PBD.

Definition 1 *An alignment of a subtrace $(\mathbf{u}_1, \mathbf{y}_1), \dots, (\mathbf{u}_m, \mathbf{y}_m)$ with a sequence of nodes $n_1, \dots, n_m$ in $(\mathcal{N}, \mathcal{E})$ is a one-to-one correspondence that maps $(\mathbf{u}_i, \mathbf{y}_i)$ to n_i . We say that the subtrace is aligned with the path, or, more simply, with the procedure structure.*

Thus, an alignment is simply a labeling of the SAPs in a trace where steps are used labels. The probability of an alignment of a subtrace with a path is computed as follows: let $\alpha_i(\mathbf{s}_i \mid \mathbf{u}_i)$ denote the conditional probability of $\mathbf{s}_i$ given $\mathbf{u}_i$ according to the action probability distribution α_i associated with n_i ; let also $\tau_i(n_i \mid \mathbf{u}_i)$ be the conditional transition probability from node n_{i-1} to node n_i given $\mathbf{u}_i$. Then the probability of the alignment is

$$\mathbb{P}\{(\mathbf{u}_1, \mathbf{y}_1), \dots, (\mathbf{u}_m, \mathbf{y}_m) \mid n_1, \dots, n_m\} = \alpha_1(\mathbf{s}_1 \mid \mathbf{u}_1) \prod_{j=2}^m [\alpha_j(\mathbf{s}_j \mid \mathbf{u}_j) \tau_j(n_j \mid \mathbf{u}_j)]; \quad (1)$$

Definition 2 A probabilistic alignment of a subtrace $(\mathbf{u}_1, \mathbf{y}_1), \dots, (\mathbf{u}_m, \mathbf{y}_m)$ with a procedure structure $(\mathcal{N}, \mathcal{E})$ is the probability distribution that assign to each sequence of m nodes the probability defined by Equation 1.

A probabilistic alignment also induces a probability distribution over the nodes for each SAP in the trace, called *alignment probability distribution*: the probability that the i th SAP $(\mathbf{u}_i, \mathbf{y}_i)$ is aligned with node n is the sum of the probabilities of the m -sequences where the n_i is n .

A *deterministic alignment* is a degenerate case of probabilistic alignment such that the action probabilities and the transition probabilities only have values zero and one.

We now define the notion of consistency of a trace with a model. Intuitively, the notion of consistency is the following. For every alignment of a trace $\{(\mathbf{u}_1, \mathbf{y}_1), \dots, (\mathbf{u}_m, \mathbf{y}_m)\}$ with the model and compute the path $n_1, \dots, n_m$ with highest probability according to Equation 1. For $i = 1$ to m , instantiate action skeleton $\mathbf{s}_i$ using the snapshot $\mathbf{u}_i$ to produce action $\hat{\mathbf{y}}_i$. If there exist an alignment for which $\hat{\mathbf{y}}_i = \mathbf{y}_i$ for $i = 1, \dots, m$, then the trace is consistent with the model.

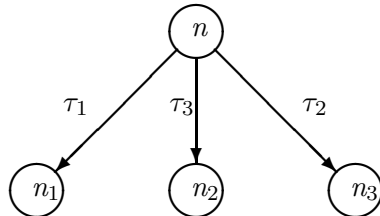
Definition 3 A subtrace $(\mathbf{u}_1, \mathbf{y}_1), \dots, (\mathbf{u}_m, \mathbf{y}_m)$ is **consistent** with a model if

- the subtrace is aligned to a sequence of nodes $n_1, \dots, n_m$, where n_i corresponds to step s_i ;

- *there exists a sequence of skeletons $\mathfrak{s}_1, \dots, \mathfrak{s}_m$ that, sequentially evaluated using $\mathbf{u}_1, \dots, \mathbf{u}_m$, yields $\mathbf{y}_1, \dots, \mathbf{y}_m$;*
- *for $i = 1, \dots, m - 1$ there is a directed edge from n_{i-1} to n_i ;*
- *the path $n_1, \dots, n_m$ attains the maximum probability according to Equation 1 evaluated with the subtrace.*
- *if the $(\mathbf{u}_1, \mathbf{y}_1)$ is the first SAP in a full procedure demonstration, s_1 must be an initial step, that is, a step without incoming edges.*

If the alignment is deterministic, then the maximum probability path is unique and has probability 1. We say that *a model is consistent* with a collection of subtraces if each subtrace is consistent with the model.

We introduce the *generalization problem* as the problem of inducing a procedure model given a procedure structure and an alignment of the training data to that structure. We need additional notation to discuss solutions to the generalization problem. For each node $n \in \mathcal{N}$, let $\mathcal{E}(n) \subseteq \mathcal{E}$ be the set of outgoing edges, let $\mathcal{N}(n)$ be the corresponding destination nodes, and let $\tau(n) \subseteq \mathcal{T}$ be the corresponding transition probabilities. For example:



In the picture, $\mathcal{N}(n) = \{n_1, n_2, n_3\}$ and $\mathcal{T}(n) = \{\tau_1, \tau_2, \tau_3\}$.

We now propose a sufficient characterization of solutions to the generalization problem.

Proposition 1 Solution to the generalization problem. *A procedure model $(\mathcal{S}, \mathcal{E}, \mathcal{T})$ is a solution to the generalization problem if it satisfies the following conditions:*

- a Each step $s = (n, \alpha)$ in $\mathcal{S}$ is induced using only SAPs having alignment probability greater than zero with n ;*
- b Consider the collection $\{((\mathbf{u}_1, \mathbf{y}_1)^i, (\mathbf{u}_2, \mathbf{y}_2)^i)\}_{i=1}^{M(n)} = \mathbf{T}(n)$ of length-2 subtraces where the alignment probability of $(\mathbf{u}_1, \mathbf{y}_1)^i$ with n and of $(\mathbf{u}_2, \mathbf{y}_2)^i$ with a node in $\mathcal{N}(n)$ is greater than zero (here $M(n) = |\mathbf{T}(n)| \geq 1$). The transition probabilities in $\mathcal{T}(n)$ are induced using only the snapshots from $(\mathbf{u}_2, \mathbf{y}_2)^i \in \mathbf{T}(n)$.*
- c The model $(\mathcal{S}, \mathcal{E}, \mathcal{T})$ is consistent with all the traces in $\mathbf{T}$.*

The alignment probabilities can in general be used in the induction process, but this is not explicitly required by the the definition. In simple terms this definition can be rephrased as follows. First, induce each step using the aligned SAPs. Then, induce the transition probabilities using the SAPs aligned with the destination nodes. If the resulting procedure is consistent with the traces, it is a solution to the problem. Thus, if a procedure

structure and a good alignment is provided, the problem of constructing a procedure model is reduced to that of instantiating steps and inferring transition probabilities. Many existing PBD systems solve only the generalization problem, and the user must provide, either directly (e.g., by labeling action) or indirectly (via feedback mechanisms), the structure and the alignment. We are only aware of systems in which the user explicitly provides a deterministic alignment. We also believe that, when the user provides the alignment indirectly via feedback, the PBD system often relies on a probabilistic alignment. Note that learning from a deterministic alignment is a simpler problem than learning from a probabilistic alignment.

An important contribution of this work is the ability to learn from multiple demonstrations when the alignment between the steps of the demonstrations is not given. When the alignment is not given, the learning algorithm must build a procedure structure using a collection of traces and align these traces to the procedure structure before solving the generalization problem. We call this the *alignment problem*. Clearly, solving the alignment problem alone does not yield a procedure model, and the learner must solve the alignment-and-generalization problem. We now define sufficient properties of a solution to the alignment-and-generalization problem.

Proposition 2 Solution to the alignment-and-generalization problem. *Given a training set $\mathbf{T}$, a solution to the alignment-and-generalization problem consists of a procedure model $(\mathcal{S}, \mathcal{E}, \mathcal{T})$, with structure $(\mathcal{N}, \mathcal{E})$, and of*

a (probabilistic) alignment of the training set to the structure, such that:

- a For each node in $\mathcal{N}$ there is at least one SAP in $\mathbf{T}$ with alignment probability greater than zero; hence, data is available to instantiate each step.*
- b For each edge $e_{i,j}$ (from n_i to n_j) there must be at least two SAPs in a trace of $\mathbf{T}$, $(\mathbf{u}_t, \mathbf{y}_t)$ and $(\mathbf{u}_{t+1}, \mathbf{y}_{t+1})$ immediately following $(\mathbf{u}_t, \mathbf{y}_t)$ such that the probability that $(\mathbf{u}_t, \mathbf{y}_t)$ is aligned with n_i and $(\mathbf{u}_{t+1}, \mathbf{y}_{t+1})$ is aligned with n_j is greater than zero; thus, data is available to infer predicate $\tau_{i,j}$.*
- c The model $(\mathcal{S}, \mathcal{E}, \mathcal{T})$ is consistent with all the traces in $\mathbf{T}$.*

Many existing PBD systems solve constrained versions of the alignment-and-generalization problem, by restricting the class of produced procedure structures, for example, by restricting attention to sequences of steps with embedded fixed-length loops, or to procedures consisting of an outer loop with a known or unknown number of steps.

3.6.2 Additional Characteristics of Learning Algorithm

In this subsection we discuss additional characteristics of learning algorithms for PBD. Some of these are of general importance, like prediction ability, while others are more relevant in certain usage scenarios (e.g., transparency is more important when the PBD system is used for guidance than when it

is used for automation).

Prediction ability: the learning algorithm must be able to predict the correct sequence of actions given a sequence of snapshots. When multiple alternative actions are appropriate, the learning algorithm should be able to assign them an appropriateness score. A typical example is a procedure containing subtasks that can be executed in any order. At the point where these tasks must be executed, the learning algorithm should favor the most common order, that is, it should give higher score to the specific task that is on the “well-worn path” learned from the executions.

Transparency: the learning algorithm should induce a procedure model that can be understood by the user. More specifically, the entire procedure structure, with decision points and loops, should be easily derivable from the learned model. Additionally, individual decision points and actions should be explainable to the user using information derived from the model.

Support for variabilization: the learning algorithm should be able to abstract string literals into variables. For sub-procedures repeated on multiple items, the learning algorithm should variabilize the objects with which the user interacts, so that actions are performed on the “current” item rather a specific item.

In general, these characteristics are in contrast with each other, and one must address the resulting trade-offs when selecting or designing a learning algorithm. For example, prediction ability can in general be improved by choosing powerful probabilistic models over deterministic models; however, probabilistic models are very difficult to visualize in a human-readable form, and therefore the transparency is generally limited.

3.6.3 The Learning Algorithm of the Sheepdog System

We now describe SimIOHMM, the learning algorithm used in the Sheepdog system. The main design point in designing SimIOHMM were: constructing procedure models in batch mode (to support parallel collaborative PBD), producing probabilistic models from which well-worn paths emerge, solving the alignment-and-generalization problem, providing high prediction ability, and supporting variabilization.

This section is organized as follows: we first introduce the general class of probabilistic models to which SimIOHMM belongs; we then describe the IOHMM, the main precursor of the SimIOHMM; we formally introduce the SimIOHMM, show how the SimIOHMM fits into the procedure model framework described in the previous section, provide an intuitive argument describing how it solves the alignment-and-generalization problem, and conclude with some implementation notes.

Procedure models as probability measures. A Sheepdog procedure

is a probability measure $\mathbb{P}\{\cdot\}$ over finite sequences of snapshot-action pairs $\{(\mathbf{u}_t, \mathbf{y}_t)\}_{t=1}^T$. We have chosen the probability framework to model the uncertainty arising from the variability of system configurations and of user preferences, and from unpredictable circumstances. In our description, as customary, we use capital letters to denote random variables, lower case letters to denote values or indices such as time (the meaning will be clear from the context), boldface letters to denote vectors, and roman typeface to denote scalar quantities.

We can write the probability of a finite random sequence $\{(\mathbf{U}_t, \mathbf{Y}_t)\}_{t=1}^T$ using the chain rule as

$$\mathbb{P}\left\{\{(\mathbf{U}_t, \mathbf{Y}_t)\}_{t=1}^T\right\} = \prod_{k=1}^T \mathbb{P}\left\{\mathbf{Y}_k \mid \mathbf{U}_k, \{(\mathbf{U}_t, \mathbf{Y}_t)\}_{t=1}^{k-1}\right\} \cdot \mathbb{P}\left\{\mathbf{U}_k \mid \{(\mathbf{U}_t, \mathbf{Y}_t)\}_{t=1}^{k-1}\right\},$$

which explicitly shows the dependence of the snapshot at time k on the previous snapshot-action pairs and of the action at time k on the snapshot at time k and on the previous snapshot-action pairs.

The goal is to learn from the training data a stepwise prediction rule $\hat{\mathbf{y}}_k = \hat{\mathbf{y}}_k\left(\mathbf{u}_k, \{(\mathbf{u}_t, \mathbf{y}_t)\}_{t=1}^{k-1}\right)$, that predicts the action to be taken next given the current snapshot and the past snapshot-action pairs. Since we assume that sequences of snapshot-action pairs are generated according to a probabilistic models, we also write $\hat{\mathbf{Y}}_k = \hat{\mathbf{y}}_k\left(\mathbf{X}_k, \{(\mathbf{U}_t, \mathbf{Y}_t)\}_{t=1}^{k-1}\right)$ to indicate that the result of the prediction is a random variable (since it is a function of random variables). The function $\hat{\mathbf{y}}_k(\cdot, \cdot)$ is then chosen to minimize the probability of incorrect prediction: $\mathbb{P}\left\{\hat{\mathbf{Y}}_k \neq \mathbf{Y}_k\right\}$. Classical Bayes Decision Theory

ensures that

$$\mathbf{Y}_k^* = \arg \max_{\mathbf{y} \in \mathcal{Y}} \mathbb{P} \left\{ \mathbf{y} \mid \mathbf{U}_k, \{(\mathbf{U}_t, \mathbf{Y}_t)\}_{t=1}^{k-1} \right\}$$

minimizes the probability of action prediction error, and therefore the problem of learning a procedure with respect to an error minimization criterion can be rephrased in terms of learning the probabilistic model $\mathbb{P} \{ \cdot \}$ from the data. The general problem of inferring a probability measure on time series is a daunting task, and further assumptions must be made to ensure that it is treatable.

The following consideration was instrumental in helping us select a class of models: when dealing with procedural knowledge, it is reasonable to assume that the next action made by the user depends on the current state of the system and on the user’s notion of the “current point within the procedure,” rather than on the specifics of all the individual past snapshot-action pairs. For example, a user who is organizing a meeting might decide to send out meeting invitations via e-mail after having successfully found an appropriate room and set up a teleconference for off-site participants: the motivation of the next action therefore does not depend on the specific details of all snapshot-action pairs observed in the past, but rather on a succinct description of their meaning and consequences. This description is meant to capture the user’s notion of “current point” within the procedure. Thus, it is a representation of the notion of step introduced in the previous section.

Clearly, the approach of modeling the observations as independent and identically distributed random variables does not capture the just-described constraints. A similar consideration holds for m -th order Markov Models: they fail to capture long-term dependences if the order m of the model is small, and are too expensive to induce if the order is large, since the cost of inference grows exponentially in m .

A class of models that capture our insights on the structure of procedures are the Input/Output Hidden Markov Models, or IOHMMs [5, 3], stochastic Mealy machines that approximate $\mathbb{P}\{\cdot\}$ by adding a *hidden variable* $\mathbf{X}$ (the “state”), belonging to a finite set $\mathcal{X}$. Recall, briefly, that stochastic Mealy Machines are finite-state automata where the input $\mathbf{u}_t$, the output $\mathbf{y}_t$, the state $\mathbf{x}_t$ at time t , and the state $\mathbf{x}_{t-1}$ at time $t - 1$ are related by the pair of equations:

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_t)$$

$$\mathbf{y}_t = g(\mathbf{x}_t, \mathbf{u}_t).$$

Bengio and Frasconi’s IOHMM. The goal of an IOHMM is to model the *conditional* probability of an output sequence $\{\mathbf{Y}_t\}_{t=1}^T$ given an input sequence $\{\mathbf{U}_t\}_{t=1}^T$ as a Hidden Markov Model. In our system, snapshots play the role of inputs, and the two terms are used interchangeably.

$$\mathbb{P}\left\{(\mathbf{U}_t, \mathbf{Y}_t, \mathbf{X}_t) \mid \{(\mathbf{U}_j, \mathbf{Y}_j, \mathbf{X}_j)\}_{j=1}^{t-1}\right\} = \mathbb{P}\{(\mathbf{U}_t, \mathbf{Y}_t, \mathbf{X}_t) \mid \mathbf{X}_{t-1}\}.$$

The same functional structure is also inherited by the conditional probability

of the outputs given the inputs which we can also write as

$$\mathbb{P}\left\{(\mathbf{Y}_t, \mathbf{X}_t) \mid \mathbf{U}_t, \{(\mathbf{U}_j, \mathbf{Y}_j, \mathbf{X}_j)\}_{j=1}^{t-1}\right\} = \mathbb{P}\left\{(\mathbf{Y}_t, \mathbf{X}_t) \mid \mathbf{U}_t, \mathbf{X}_{t-1}\right\}.$$

Note that we can further decompose this conditional probability as

$$\mathbb{P}\{\mathbf{X}_t \mid \mathbf{U}_t, \mathbf{X}_{t-1}\} \mathbb{P}\{\mathbf{Y}_t \mid \mathbf{X}_t, \mathbf{U}_t, \mathbf{X}_{t-1}\} = \mathbb{P}\{\mathbf{X}_t \mid \mathbf{U}_t, \mathbf{X}_{t-1}\} \mathbb{P}\{\mathbf{Y}_t \mid \mathbf{X}_t, \mathbf{U}_t\},$$

which is the product form of the conditional transition probability to state $\mathbf{X}_t$ given the previous state, and the input at time t , and the conditional probability of the output at time t given the state and input at time t . This decomposition shows that inducing a IOHMM is equivalent to estimating the initial probability measure over the states, $\mathbb{P}\{\mathbf{X}_0\}$ (where we use the convention that the first input is observed at $t = 1$), the transition probabilities $\mathbb{P}\{\mathbf{X}_t \mid \mathbf{U}_t, \mathbf{X}_{t-1}\}$, and the conditional output probabilities $\mathbb{P}\{\mathbf{Y}_t \mid \mathbf{X}_t, \mathbf{U}_t\}$.

We briefly pause to note that, in Sheepdog, the outputs of the IOHMM are really action skeletons that are evaluated to actions using the inputs and context information. However, in the interest of simplicity, we will treat the outputs as if they were actual actions and use the corresponding notation.

The transition and output probabilities are assumed to be time-independent, that is, for every s and t ,

$$\begin{aligned} \mathbb{P}\{\mathbf{X}_t = a \mid \mathbf{U}_t = \mathbf{u}, \mathbf{X}_{t-1} = b\} &= \mathbb{P}\{\mathbf{X}_s = a \mid \mathbf{U}_s = u, \mathbf{X}_{s-1} = b\} \quad \text{and} \\ \mathbb{P}\{\mathbf{Y}_t = \mathbf{y} \mid \mathbf{U}_t = \mathbf{u}, \mathbf{X}_t = \mathbf{x}\} &= \mathbb{P}\{\mathbf{Y}_s = \mathbf{y} \mid \mathbf{U}_s = \mathbf{u}, \mathbf{X}_s = \mathbf{x}\}. \end{aligned}$$

These assumptions make the IOHMM very flexible and yet computationally manageable. The IOHMM allows arbitrarily long time-dependence between

the input-output pairs, and is therefore more powerful than a fixed-order Markov model. At the same time, the IOHMM can be efficiently induced from training data using the Baum-Welch algorithm, consisting of two steps: an Expectation step, where the training sequences are aligned to an existing model, and a Maximization step, where the model is updated given the alignment.

The IOHMM E-step efficiently computes a probability measure over the state space for each snapshot-output pair by means of two steps: the forward recursion computes, for each time t the joint probability of the state at time t and all outputs up to time t given the inputs up to time t ; the backward recursion computes the probability of the outputs after time t , given the state at time t and the inputs from time t on. The results of the forward and backward recursions are then appropriately combined to estimate the probability measure over the states as well as the posterior transition probabilities. For the details, we point the interested user to Bengio and Frasconi’s cited papers, whose notation we follow in this section.

The M-step efficiently recomputes the initial probability measure over the states, the conditional transition probabilities given the current input, and the conditional probability measure of the output given the current state and input, either by maximum likelihood or with a generic likelihood estimation method. Bengio and Frasconi followed the latter approach, and used neural networks in the M-step. The efficiency of the M-step stems

from the fact that the *global* maximization of the likelihood is performed by *separately* maximizing the likelihoods at each individual state, which is attained by finding the parameters that maximize the transition and output probabilities for each state given the results of the expectation step.

The SimIOHMM. The SimIOHMM extends the IOHMM by further incorporating a new source of bias: similarity between SAPs. More specifically, SimIOHMM associates one or more representative SAPs with each state of the hidden Markov chain. The representative SAPs are used during the E-step to compute the probabilities and are updated during the M-step. Intuitively, a training SAP “attracted” towards Markov states whose representative SAPs are “similar” to it, and away from Markov states whose representative SAPs are “dissimilar”.

The E-Step. The forward recursion is

$$\alpha_{i,t} = \mathbb{P}\{\mathbf{y}_t \mid \mathbf{X}_t = i, \mathbf{u}_t\} \mathbb{S}(\mathbf{u}_t, \mathbf{v}_i) \sum_{\ell \in \mathcal{X}} \phi_{i,\ell}(\mathbf{u}_t) \alpha_{\ell,t-1}, \quad (2)$$

which adds to [3, Equation(20)] the additional term $\mathbb{S}(\mathbf{u}_t, \mathbf{v}_i)$, the similarity score between the input $\mathbf{u}_t$ and the representative sample $\mathbf{v}_i$ of state i . $\mathbb{S}(\mathbf{u}_t, \mathbf{v}_i)$ is the SimIOHMM bias. In the equation, the ϕ are the transition probability and the α are the forward probabilities. We refer the unfamiliar reader to [6] for the details of the IOHMM forward recursion. Similarly, the backward recursion is

$$\beta_{i,t} = \sum_{\ell \in \mathcal{X}} \mathbb{S}(\mathbf{u}_{t+1}, \mathbf{v}_\ell) \mathbb{P}\{\mathbf{y}_{t+1} \mid \mathbf{X}_{t+1} = \ell, \mathbf{u}_{t+1}\} \phi_{\ell,i}(\mathbf{u}_{t+1}) \beta_{\ell,t+1}, \quad (3)$$

which again adds a bias term to [3, Equation(22)].² The β terms are backward probabilities in the recursion.

The bias term $\mathbb{S}(\mathbf{u}_t, \mathbf{v}_i)$, a similarity score, is designed to concentrate the state probability measure on states having representative samples similar to $\mathbf{u}_t$. For sake of simplicity, assume that each state has a unique representative sample. Then $\mathbb{S}(\mathbf{u}_t, \mathbf{v}_i)$ is computed as follows: first, the distances $\{d(\mathbf{u}_t, \mathbf{v}_i)\}_{i=1}^{|\mathcal{X}|}$ between $\mathbf{u}_t$ and the representatives of the states are computed; then these distances are converted into a similarity score by means of a kernel $K(\cdot)$ (for example, a finite-support decreasing function or a Gaussian). Finally, the resulting values are normalized to yield similarity scores:

$$\mathbb{S}(\mathbf{u}_t, \mathbf{v}_i) = \frac{K(d_i)}{\sum_{\ell=1}^{|\mathcal{X}|} K(d_\ell)} \cdot \mathbb{S}(\mathbf{u}_t, \mathbf{v}_i) = K(d(\mathbf{u}_t, \mathbf{v}_i)).$$

The definition of $\mathbb{S}(\cdot, \cdot)$ can be obviously extended to capture the similarities between snapshot-output pairs, rather than between snapshots. Adding the bias term substantially improves the training time and can improve the classification accuracy, as illustrated in the experiments section of [6].

The M-step. The M-step of the SimIOHMM consists of the M-Step of the IOHMM, plus the re-computation of representative samples. For each HMM state, the sample with highest alignment probability becomes the new representative.

²The small discrepancies with the actual Equation [3, Equation(22)] are due to typographical errors in the original paper.

The SimIOHMM as a procedure model. The SimIOHMM can be easily interpreted as a procedure model in the sense of Section 3.6.1. The hidden states can represent nodes in the procedure graph. The SimIOHMM can be used to produce user actions, and in this case the conditional probability measures on the outputs given the input (snapshot) of each node is equivalent to the action probability measure described in the previous section. A hidden state combined with its output probability measure can be interpreted as a procedure step. Since the SimIOHMM associates representative SAPs to each hidden state, this interpretation of hidden states as nodes of procedure steps has semantic meaning (the same is not true for the IOHMM). Finally, each hidden state has a transition probability measure on the outgoing edges.

The SimIOHMM and the alignment-and-generalization problem.

The SimIOHMM solves the alignment-and-generalization problem in an iterative fashion:

During the E-step, the SimIOHMM uses the procedure model $\mathcal{M}$ generated during the previous iteration of the E-M algorithm to align the traces onto the procedure graph; the similarity bias ensures that dissimilar SAPs are not grouped together. As part of the E-step, SimIOHMM constructs new nodes whenever it encounters SAPs that are not aligned with any step of $\mathcal{M}$, and discards steps for which the alignment prob-

ability with every SAP is zero. Hence SimIOHMM solves both the alignment and the graph-structure selection problems by relying on $\mathcal{M}$.

During the M-step, SimIOHMM uses the alignment of the traces produced by the E-step to solve the generalization problem, that is, to construct the conditional action and transition probabilities given the snapshot.

Thus, SimIOHMM identifies a solution by alternating the steps of constructing the best alignment of the data for a given model, and constructing the model that best explains the alignment.

Implementation notes. The SimIOHMM is implemented using classifiers to estimate the transition and output probabilities. Except in unrealistically simple examples, it is not possible to use tables to represent the conditional transition and output probabilities given the snapshots, as it is often done in classical HMMs, because one needs a separate table per each state *and* each configuration of the snapshot vector. The number of tables therefore grows exponentially with the length of the feature vector, making the brute-force tabular method useless.

The Sheepdog implementation uses the Weka [7] J48 decision trees. This choice has two motivations: a user performing a procedure typically makes decisions based on a small number of parameters (e.g., the decision to check a check-box depends on whether it is unchecked or not, and ignores other on-

screen features), and decision trees are well-suited for learning these types of concepts. Furthermore, decision trees can be inspected to explain how the algorithm produces its predictions. We have also experimented with a variety of other classifiers, including Naive Bayes, other types of decision trees, and k -nearest-neighbors algorithms; we observed that none of the explored alternatives offers appreciably higher classification accuracy for our class of problems.

We have experimented with a variety of kernels, and concluded that smooth kernels with finite support outperform non-smooth kernels and kernels with infinite support. However, we have not observed a statistically significant difference between different smooth kernels. The implementation used in the experiments relies on a quadratic kernel.

Limitations. There are two main limitations to the SimIOHMM.

- If the current action is not conditionally independent of past actions and snapshots given the current snapshot and hidden state, the algorithm might fail to learn it correctly. This limitation is a consequence of the HMM approximation to the procedure model. Although this has not proved to be a problem in the test cases we have used so far, we can conceive procedures where the HMM approximation can lead to incorrect predictions. We are considering different approaches to solving this problem, including selectively maintaining a history of

features obtained from previous snapshots.

- The number of examples required to learn a concept (for example, a branch point) varies with the learning task in a fashion that is not a-priori characterizable. This is an intrinsic limitation of statistical classifiers for which there is no universal convergence rate to the *true* concept³. By using a set of heuristics, we have successfully reduced the dimensionality of the feature vectors describing the state of the system. However, as with any statistical classifier, we have little control over which features the classifier automatically selects to produce a rule, and on how many examples are required to learn the correct rule with high probability. We realize that, when dealing with complex user interfaces, the number of required examples could be very large, and we are considering allowing the expert to add side information by explicitly indicating the relevant features for selected procedure steps.

3.7 Authoring in Sheepdog

Sheepdog is designed to operate with any application running on the Windows operating system. As a consequence, it does not use application-specific knowledge, and it cannot automatically generate semantic descrip-

³The authors of one of the most comprehensive and most technical books on statistical pattern recognition, “A Probabilistic Theory of Pattern Recognition”[8], discuss this matter in Chapter 1, and conclude “For this reason, with few exceptions, we will steer clear of the rate of convergence quicksand”.

tions at the execution step or sub-procedure levels. Since we believe in the importance of providing a description of the task at semantic levels higher than the action level, we have implemented an authoring facility that can be used to organize the structure of the procedure in a semantically meaningful fashion for the benefit of the user.

After inducing a procedure from demonstrations, the Sheepdog learning algorithm constructs a simple description in terms of user actions. Via the authoring interface, the author can hierarchically group these action descriptions. The author can also manually add annotation to these groups as well as individual actions. Additionally, the author has the option of visually distinguishing between *mandatory* and optional *actions*, that is, between actions that need to be performed by all users to successfully complete the procedure, and actions that are not important to the success of the procedure and that users can decide to perform or discard (consider for example a procedure to install an application where the user can optionally change the default font size). Finally, the author explicitly marking decision points (branch points), that is, places in the procedure where the actions to be taken next depend on the specific environment of the user (e.g., a configuration task where certain steps depend on the OS, say Windows 2000 vs. XP). A user executing the procedure is presented with a graphical description of its structure (Figure 4). User actions that are grouped by the author are represented by a folder icon, while individual steps are marked by a “page”

icon. The user can explore a group of actions by recursively opening the corresponding folder.

Authoring is separate from automatic abstraction and inference: the learning algorithm produces its best guess of the procedure structure given the actions recognized by the abstraction layer, and is not influenced by the manual authoring. We have since constructed a new incremental learning algorithm, called Augmentation-Based Learning [9], that relies on both recorded traces and manual authoring while constructing an executable procedure model. Augmentation-Based Learning is not used by Sheepdog.

3.8 Execution

The execution engine performs the procedure on behalf of or in cooperation with the user. The following are desiderata for the execution engine and user interface.

Emulation of individual user actions: the execution engine must be able to interact with different types of widgets to emulate the actions predicted by the procedure model.

Robustness to variations in the target application GUI: the execution engine must be able to identify correctly the *targets* of the actions (i.e., the UI components on which the action is performed). This requirement is substantially more important in distributed collaborative PBD than in

traditional PBD, because UI differences that occur in a single machine are typically less dramatic than across machines, and consequently an algorithm that succeeds in identifying the correct targets on a single machine need not be successful across different machines.

Automatic execution of a procedure at different granularity: the execution engine should be able to execute individual steps as well as sequences of steps on behalf of the user. The granularity of execution could be different in different parts of the procedure, and the user should be allowed to change it during execution (for example, the user should be allowed to step through the first part of the procedure and ask the system to perform automatically the rest).

Support for explanation of the procedure: the execution interface should provide the user with a description of the entire procedure and of the current step.

Support for tutoring mode: the UI should support a mode where the user is guided through the procedure: the system suggests appropriate next actions, but the user is required to perform the actions on the target application. To this end, the UI should have provisions for suggesting the next step, for describing how it should be performed, and for highlighting the widgets where the action is to be taken.

Support for mixed-initiative mode, namely, for a mode of interaction where the initiative is seamlessly exchanged between the user and the system. More specifically, the execution engine should detect actions performed by the user and relinquish the initiative accordingly. The execution engine should switch to a mode similar to the tutoring mode, where it detects user actions, updates the state of the procedure model model, predicts the next action, and reports the prediction to the user. The UI should provide a facility for the user to relinquish the initiative back to the execution engine. An example of mixed initiative interaction was described in Section 2.

Ability to detect completion of an action: the execution engine must wait until an action has completed before attempting to perform the next action. More specifically, the effects of an action often influence the decision of which action to perform next. Thus, the execution engine should not attempt to predict the next action before the effects of the current action are visible on the screen.

3.8.1 Sheepdog's Execution Engine and Execution GUI

Sheepdog can automatically execute learned and authored procedures. In its simplest mode, Sheepdog automatically executes the entire procedure. However, Sheepdog has additional facilities that allows it to execute a procedure in cooperation with the user. For this purpose, Sheepdog has a Execu-

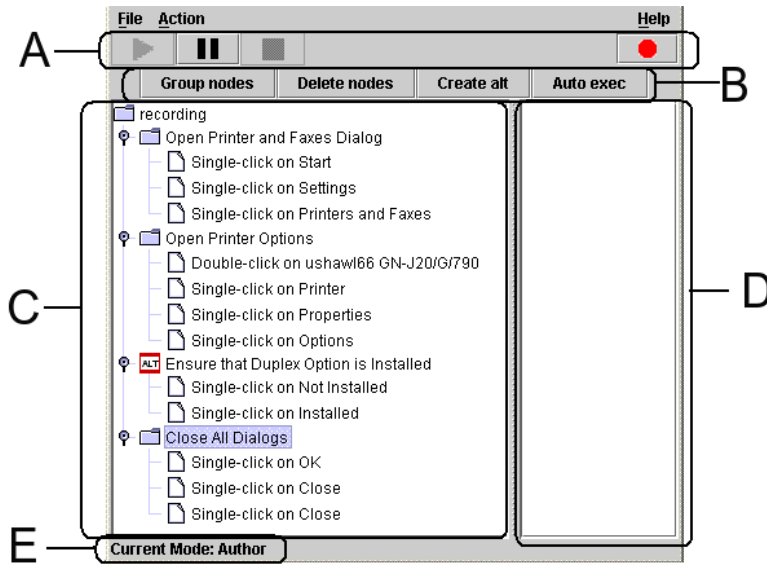


Figure 8: Components of the Sheepdog GUI. A: VCR-style controls. B: mode-specific buttons. C: procedure pane. D: step annotation pane. E: indicator of current mode.

tion GUI mode which describes the structure of the procedure, graphically guides the user through the execution, and explains individual steps.

The Execution GUI mode, depicted in Figure 8, loads the two components of an authored procedure, namely, the executable procedure model produced by the learning algorithm, and the manually authored procedure representation (Section 3.7). The authored procedure representation is graphically depicted in the procedure pane, on the left-hand side of the Execution GUI mode, and reflects the results of authoring. The Execution Engine requests a snapshot of the system from the instrumentation. This snapshot includes a description of the existing windows and the value of

registry and other system variables. This description is preprocessed and provided as input to the executable procedure model. The executable procedure model produces a description of the next action to be performed, which includes means of identifying the widget on which to act, a short description of the action, and, if available, annotations provided by the author. The Execution GUI mode then highlights the action description on the left-hand side of the panel, displays annotations as appropriate on the right-hand side of the panel, and flashes the window or widget where the interaction will take place. In short, the Execution GUI mode guides the user through the individual steps, describing how to interact with the relevant applications and widgets. Currently, the Authoring GUI provides the author with the ability to specify the granularity at which the procedure is executed. The user can override these defaults by switching to the authoring mode.

We describe separately the execution of the first action in the procedure and the execution of the subsequent ones. Executing the first action is simpler than executing any the subsequent ones. One of the reasons is that each action performed on an application can have effects that are not immediately visible in the UI. An example is launching an application by double-clicking an icon on the desktop: the time interval between the double-click and the instant where the application is available for interaction can be several seconds long, and can vary substantially depending on the application, the characteristics of the computer, and the current processing load. Recall how

the HMM models a procedure step: based on the current snapshot (describing the content of the target application GUI) and state (i.e., distribution over the states) the HMM first performs a state transition (i.e., selects a new distribution over the states), then infers an action. Each step is performed only after the effects of the previous actions have been observed. Hence, the Execution Engine must decide when the previous action has completed and its effects observed.

The current implementation of the Execution Engine relies on two heuristics to decide when it is allowed to take the next step: *quiescence* and *preconditions*. When all the effects of a user action are reflected on an application GUI, the application GUI reaches quiescence. Preconditions are requirements on the application GUI that need to be satisfied for an action to be performed (for example, the target of the action must be visible). Quiescence is detected by means of a timer that is reset when a change to the GUI is observed. When the timer fires, the Execution Engine declares that the target application GUI is quiescent. The Execution Engine then snapshots the application GUI, uses the snapshot to infer the next action, and checks the preconditions. If quiescence and preconditions are satisfied, the Execution Engine selects the next step.

The discussion of this section highlights the first main difference between a macro recorder and the Sheepdog Execution Engine. A macro recorder reproduces a sequence of recorded actions. The Sheepdog Execution Engine

relies on the procedure model to dynamically and adaptively select the next action to be taken using the current content of the GUI and requirements on the content of the GUI imposed by the possible next steps.

The second difference is an ability to execute a procedure in mixed initiative mode, described in the next subsection.

3.8.2 Executing in Mixed Initiative Mode

In mixed initiative mode (also called collaborative execution mode), the system allows the user to take the initiative, that is, to manually perform actions. The user is allowed both to continue to perform the procedure (perhaps in a different order from the one selected by the Execution Engine) and to perform actions that are extraneous to the procedure—for example, writing an e-mail or responding to a message.

We distinguish between *on-track* and *off-track* mixed initiative. In on-track mixed initiative, the user performs sequences of action that are consistent with the procedure model, while in off-track mixed initiative the user deviates from the execution. Deviation can occur for a variety of reasons: for example, the user can temporarily switch to a different task, such as responding to an instant message; alternatively, the user can decide to take a path through the process that is not captured by the procedure model; finally, the user can respond to unforeseen circumstances, such as an error condition.

While the user retains the initiative, the Execution Engine matches the observed user actions with the model, as described below. The user is deemed to be on track if the match is successful. In this case, after matching each observed action, the Execution Engine infers the next action to be taken and suggests it to the user by highlighting it in the Sheepdog GUI. This provides both a visual cue that the Execution Engine is following along, and guidance on what should be done next, which can be useful in a variety of situations, including training novice users.

Matching the observed user actions with the model is done using the well-known Viterbi Algorithm [10]. The Execution Engine decides that the user actions are consistent with the HMM if the probability of the most likely path is sufficiently high (compared to the probabilities of the most likely paths of the training traces), if the observed transitions have non-zero probability, and if the user actions can be matched to actions skeletons associated with the aligned steps. If any of the above conditions fails—and in particular if the user performs an action that cannot be matched to action skeletons—the Execution Engine decides that the user is off-track, notifies the user, and does not attempt to immediately regain the initiative.

When the user is deemed to be off-track, the Execution Engine analyzes the most recently observed actions by aligning them to the procedure model and computing the corresponding alignment probabilities. More specifically, if $(\mathbf{u}_0, \mathbf{y}_0)$ is the most recent snapshot-action pair, $(\mathbf{u}_{-1}, \mathbf{y}_{-1})$

is the previous one, and so on, the Execution Engine computes the best alignment probabilities of the $h + k - 1$ subsequences $[(\mathbf{u}_{-1}, \mathbf{y}_{-1}), (\mathbf{u}_0, \mathbf{y}_0)]$, $[(\mathbf{u}_{-2}, \mathbf{y}_{-2}), (\mathbf{u}_{-1}, \mathbf{y}_{-1}), (\mathbf{u}_0, \mathbf{y}_0)]$, $\dots$, $[(\mathbf{u}_{-k-h}, \mathbf{y}_{-k-h}), \dots, (\mathbf{u}_0, \mathbf{y}_0)]$, where k is a parameter denoting the minimum number of on-track actions before re-alignment is declared, and h is an appropriate constant further described below. Let p_1 to p_{k+h} denote these best alignment probabilities. If the user is entirely off-track, then p_1 to p_{k+h} are small. As the user starts performing a sequence of actions that are consistent with the procedure model, the first probabilities become large. The Execution Engine decides that the k most recent user actions are on track and that the previous ones are off-track if it detects a discontinuity (technically, a change point) between the probabilities of the most recent k actions and of the most recent $k + 1$ actions, and if there is no additional, more recent change point. When the Execution Engine decides that the user is back on track, it resumes operations as described above.

3.8.3 Finding the right widget

A very difficult problem that arises in collaborative PBD is the correct identification of the action targets. Environmental differences, including difference between versions of the OS or of applications, user settings, and environmental configuration can make the task of identifying a target widget challenging. Sheepdog associates with each target widget a description that

includes type, geometry, text, and other attributes of the widget itself, as well as hierarchical information on the containing widgets and their state, and on the application that owns the widget. This information is used during playback to identify the widget, as described in Figure 9.

The matching algorithm uses a collection of hard constraints on the characteristics of the target widget, $\mathcal{H}$, and a collection of soft constraints, $\mathcal{S}$. A constraint is hard if the desired search target can never violate it. A constraint is soft if the search target is allowed to violate it. Examples of hard constraints are the type of the target widget (e.g., a toolbar button cannot be the result of a search for a menu item) and the application name (e.g., the “File” menu of a web browser cannot be the result of a search for the “File” menu of a specific word processor). Examples of soft constraints are the position relative to the containing main window, the collection of ancestor windows (which can change if the widget is in a movable container, or across versions of the same application), the associated icon (for example, Microsoft Windows XP allows the user to choose between a “classic” and an “XP” look), the index in the containing window (for example, Microsoft Word dynamically reconfigures menus depending on usage, and the index of the menu items varies accordingly).

When searching for a widget, Sheepdog first tries to match the widget description exactly, using a greedy algorithm. If exact match fails, an approximate matching algorithm is invoked. The approximate algorithm

```

let  $\mathfrak{d}$  be the description of the target widget
let  $\mathfrak{a}$  the application owning the target widget
let  $\mathcal{H}$  be a set of hard constraints
let  $\mathcal{S}$  be a set of soft constraints

if a unique widget  $\mathfrak{t}$  exists that matches exactly  $\mathfrak{d}$ 
    return  $\mathfrak{t}$ 
else
    let  $\mathbb{C}$  be a set of candidate results, initialized to  $\emptyset$ 
    find the set  $\mathcal{W}$  of top-level windows owned by  $\mathfrak{a}$ 
    for each  $\mathfrak{w} \in \mathcal{W}$ 
        for each widget  $\hat{\mathfrak{t}} \in \mathfrak{w}$ 
            if  $\hat{\mathfrak{t}}$  satisfies all constraints in  $\mathcal{H}$ 
                add  $\hat{\mathfrak{t}}$  to  $\mathbb{C}$ 
            else
                discard  $\hat{\mathfrak{t}}$ 

    if  $\mathbb{C}$  is empty
        declare failure
    else if  $\mathbb{C}$  contains a single entry  $\mathfrak{t}$ 
        return  $\mathfrak{t}$ 
    else
        score each element of  $\mathbb{C}$  using the constraints  $\mathcal{S}$ 
        return the element with highest score

```

Figure 9: The widget matching algorithm.

matches available widgets to the description by means of a set of hard constraints and a set of soft constraints. Hard constraints must be satisfied exactly for a widget to be a candidate, while soft constraints need not be

satisfied. Widgets that do not satisfy the hard constraints are discarded. If one widget alone satisfies all the hard constraints, a match is declared and the widget returned. If no widget satisfies the hard constraints, a failure is declared. If two or more widgets satisfy the hard constraints, they are scored using the soft constraints.

Evaluating a soft constraint typically produces a match score. For example, matching a button within a toolbar by index produces a score that ranges from 1 (the button is in the desired position) to 0 (the button is in the farthest place from the desired position). In Sheepdog strings, such as window titles, are compared via approximate matching which yields a similarity score. Different soft constraints have different weight in the decision: matching the position of a button in a toolbar is not as critical as matching the text associated with the same button. In Sheepdog, weights were computed empirically and hard coded in the matching algorithm. However, adaptive techniques can be used that analyze the recorded states and automatically identify the relative relevance of the different constraints for specific widgets using traditional measures of discrimination (for example, the mutual information between the attribute and the widget identity). The soft constraints are finally combined by adding their scores, and the widget with highest score is returned.

During playback, the matching algorithm is used for action prediction, for display, and for execution. The next action to be executed is predicted by

the SimIOHMM, which uses the attributes of the visible widgets to select the next HMM state. Given the selected state, the SimIOHMM uses the widget attributes to select the action to perform. Therefore, prediction is predicated on correctly identifying the widgets involved in the prediction task across environments. The Execution Engine takes as input the predicted action, identifies the target widgets and, if so directed by the user, automatically interacts with them.

The matching algorithm is also used by the learning module, to align steps and learn classifiers. Alignment is predicated on matching actions from different SAPs, and the alignment score depends on whether the destination widgets are the same or different. Action and transition classifiers respectively predict the next action and HMM state from the attributes of the widgets visible to the user. An attribute of a specific widget that is relevant to the prediction task can be used effectively by the learning algorithm only if the widget can be correctly identified in the traces used for inference.

The matching algorithm of Figure 9 works very well empirically, and in our experiments we have not encountered cases in which it fails. However, the algorithm will fail to find the desired widget when the model is constructed for a version of an application and is applied to another version with radically different GUI. In particular, if the nature of a control is radically changed, the algorithm fails, such as when a control is removed from a toolbar and replaced with an item in a pull-down menu. This failure condi-

tion commonly manifests itself in GUI testing while developing applications, where the GUI can undergo radical changes during the development cycle. No solution to this problem is known. In Sheepdog the user can demonstrate the appropriate sequence of actions in the new application version, and update the procedure model with the new recording.

4 User Study

In this section, we discuss a user study that was designed with three main goals in mind: evaluating the ability of users to follow written documentation composed of text and screen shots; comparing the performance of Sheepdog trained with expert traces to that of actual users; and evaluating Sheepdog’s ability to learn from noisy data. This study was first described in [11], which contains some of the results described here.

In the study, eleven volunteers checked and corrected the network settings of a laptop computer by following written instructions obtained from the internal IT website at IBM T.J. Watson. Each volunteer was presented with a different initial configuration of the machine. An expert also carefully performed the procedure starting from each of the eleven different configurations. The actions of the volunteers and of the experts were recorded with the Sheepdog instrumentation, segmented by the abstraction layer, and stored as traces of snapshot-action pairs. At the end of each session, the subject was also administered a survey with questions on expertise and

background as well as on aspects of the experience during the session.

Section 4.1 describes the experimental settings, and Section 4.2 contains the experimental results. The analysis of the subject performance is in Section 4.2.2, while the analysis of Sheepdog’s performance is in Section 4.3.

4.1 Materials and Methods

Figure 10 depicts the flow chart of the procedure described in the written instructions to the subjects. Depending on the initial configuration of the machine, some of the depicted steps are not required, and others must be repeated more than once.

The subjects were given the written instructions, which included both text and pictures, describing an actual configuration procedure. The instructions were obtained from an internal IT web site and were printed directly from the browser. The subjects were then asked to follow the instructions as closely as possible in a controlled environment. They were told that there was no limit on the time required to complete the task. They were also presented with a laptop computer (an IBM Thinkpad model T20 with 256 Mb of main memory, running the Windows 2000 operating system equipped with a wireless network card, a model with which all subjects were familiar). Each subject had a machine with a different initial configuration of the laptop network connection, and, thus, had to execute a different path

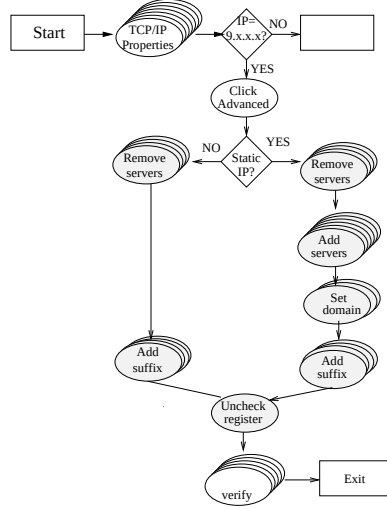


Figure 10: Network configuration procedure. Ovals represent steps in the procedure; stacked ovals represent steps that could be executed in sequence more than once; shaded ovals denote actions whose execution is required only for certain initial configurations.

through the procedure. The initial conditions were chosen to ensure that each branch point, iterated step, and optional action was exercised (e.g., for each optional action depicted in Figure 10, there is at least one initial condition that requires its execution and one initial condition that requires the step to be skipped). The subject actions were recorded by with the Sheepdog recorder (composed of the instrumentation and abstraction layers), and by a video camera with audio. A single expert also performed the procedure

starting from the same 11 different configurations, and the expert actions were similarly recorded.

The subjects were administered an exit questionnaire where they rated their familiarity with Windows 2000, the perceived clarity of the instructions, their confidence that they followed the instructions correctly, and their confidence that they produced a correct configuration of the machine. The rating used a 1 to 5 scale, where 1 indicated the lowest score and 5 the highest.

The recorded traces were analyzed manually, to investigate the performance of the individuals compared to that of the expert. Statistical analysis were performed using the statistical package of the GNU Octave software environment. Due to the small sample size, we report results obtained with the standard tests (that typically use tabulated p-values based on asymptotic distributions) as well as p-values obtained using resampling estimates (see, for example, [12, Chapter 4]).

The subject traces and the expert traces were used separately to train two executable procedure models. Each procedure model was tested against its training set and against the other set of traces. The testing methodology was the following: for each trace:

- 1 the SimIOHMM was initialized;
- 2 the snapshot part of the first snapshot-action pair in the trace was

provided to the prediction algorithm of Sheepdog;

3 the SimIOHMM predicted an action, and the prediction was recorded;

4 the SimIOHMM was then presented the first snapshot-action pair of the trace as input, which was used to update the probability distribution over the nodes;

5 steps 2 to 4 were repeated with the next snapshot-action pair in the trace, until completion of the trace;

Steps 1 to 5 were repeated for each of the traces. This approach tests the performance of the learned model when used to produce next step predictions. The prediction accuracy produced by this methodology (the ratio of the correct predictions to the total number of actions) is probably a lower bound to the predictive ability of Sheepdog when used in a mixed initiative scenario. The reason is that the procedure only tests the ability of the system to predict the next action of the specific trace, rather the ability to predict an action that is correct in the current context.

We tested the procedure learned with the expert traces against its training set. This shows how well the system learns the decision points demonstrated in the traces. We also tested the procedure learned with the expert traces against the subject traces. This shows how often an action performed by a user could not be inferred from expert demonstrations (alternatively, from the instructions). Finally, we tested a procedure learned with subject

traces against the expert traces. This provides an indication of how well the PW system can “smooth out” the noise and learn well-worn paths through the procedure.

4.2 Experimental Results

4.2.1 Survey Results

We first analyze the results of the subject survey, summarized in Table 4.2.1. In general, the users felt comfortable with the Windows environment: the mode and median of the “knowledge” entry were both 4 (“familiar”)—only 1 user selected “unfamiliar” (rating = 1), one user selected “somewhat unfamiliar” (rating = 2), while 3 users rated themselves as “expert” (rating = 5). Five users found the instructions “only moderately clear” (rating = 3), three users found the instructions “somewhat clear” (rating = 4), and three users found the instructions “perfectly clear”. Interestingly, the knowledge and clarity ratings appear to be independent: a parametric chi-squared test of independence returns a p-value of 0.527, using the asymptotic distribution for the test statistics, while a Monte-Carlo permutation chi-squared test of independence with 20000 iterations yields a p-value of 0.751 (this is probably a more accurate value given the sample size.) Similar independence results hold between knowledge and perception of having followed the instructions correctly (traditional chi-square p-value = 0.103, permutation test p-value = 0.127), as well as between knowledge and perception of having successfully

Subject	Knowledge	Clarity	Followed	Configured
1	5	5	4	5
2	4	3	5	5
3	5	5	5	5
4	5	3	4	1
5	3	3	4	2
6	3	4	4	4
7	1	3	2	3
8	4	4	4	4
9	4	4	4	4
10	4	5	5	5
11	2	3	4	4

Table 1: The result of the entry survey, by subject. Each subject selected, on a scale from 1 (lowest) to 5 (highest), the knowledge of the Windows environment, the perceived clarity of the instructions, the belief that they literally followed the instructions, and the belief that they actually produced the desired configuration.

configured the machine (traditional chi-square p-value = 0.129, permutation test p-value= 0.145). In other words, expert users did not find the instructions more or less difficult to understand and to follow than novice users.

Similarly, we observed independence between the perceived clarity and the confidence of having correctly followed the instructions (traditional chi-square p-value = 0.299, permutation test p-value = 0.364). Interestingly, there is some evidence of dependence between the perceived clarity and the confidence of having correctly configured the machine (traditional chi-square p-value = 0.102896, permutation test p-value = 0.0305: the traditional test would not reject the hypothesis of independence at the 5% significance level, but the permutation test would, although not too assertively).

4.2.2 Subject and Expert Performance

Table 4.2.2 summarizes the performance of the subjects, indexed by subject number (listed in the first column). We distinguish between (1) on-track actions, that is, actions performed correctly as described by the instructions; (2) off-path actions, that is, actions not described by the instructions and that were performed as a consequence of misinterpretation of the instructions (the off-path count include actions that were performed to correct mistakes); and (3) spurious actions, that is, actions that are not described in the instructions, that do not indicate misinterpretation on the part of the subject, and that do not negatively affect the execution of the procedure except in terms of efficiency (for example, moving and resizing a window). The Total count in column 6 is the sum of these three counts. Occasionally, the subjects missed steps described in the procedure. These are indicated in

column 5. The total number of actions performed by a subject can be easily compared to the number of actions of the expert performing the same task, which appears in Column 7. Not all the subjects successfully completed the task. Success or failure is marked in Column 8 of the table.

The success rate of the subjects was 64%; four subjects did not complete the required task successfully. Interestingly, the familiarity of the user with Windows appears to be independent of the success in completing the task (the p-values of the parametric chi-squared test is 0.396, while the p-value of the permutation chi-squared test is 0.600). Similarly, the user confidence of having correctly configured the machine appears to be independent of the success in completing the task.

The expert performed the procedure as described in the written documentation. In spite of the fact that written instructions were available, and that no time constraints were put on the exercise, no single subject managed to perform the steps exactly as described

Figure 11 illustrates this statement by showing the number of steps that subjects needed to complete the procedure together with the number of steps that the expert required when starting from the same initial configuration. Additionally, for each initial configuration (equivalently, subject) the number of actions performed by the subject that were not performed by the expert are displayed. The configurations are sorted in decreasing order of the number of steps performed by the user.

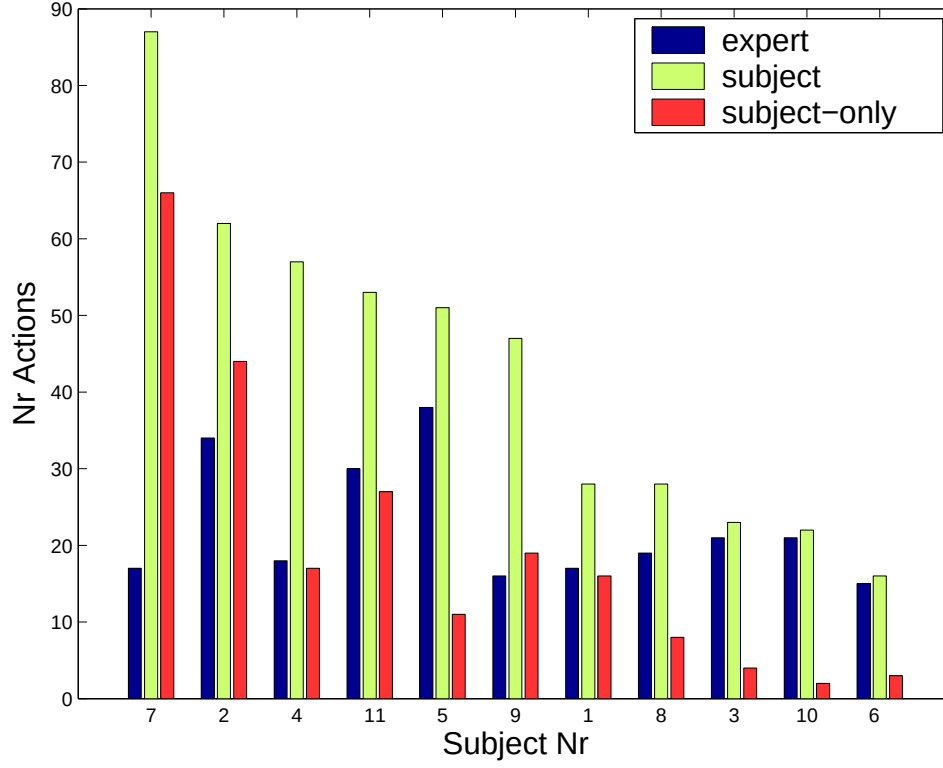


Figure 11: Number of actions performed by experts and by subjects as a function of the initial configurations. The subject-only bar counts the number of actions that were executed by each subject and did not appear in any of the expert traces. The subjects are sorted by number of performed actions.

4.3 Sheepdog Performance Study

Figure 12 illustrates the error rates for the SimIOHMM trained with the expert data and tested on the training set and on the subject data, and for the SimIOHMM trained on the subject traces and tested on the expert data.

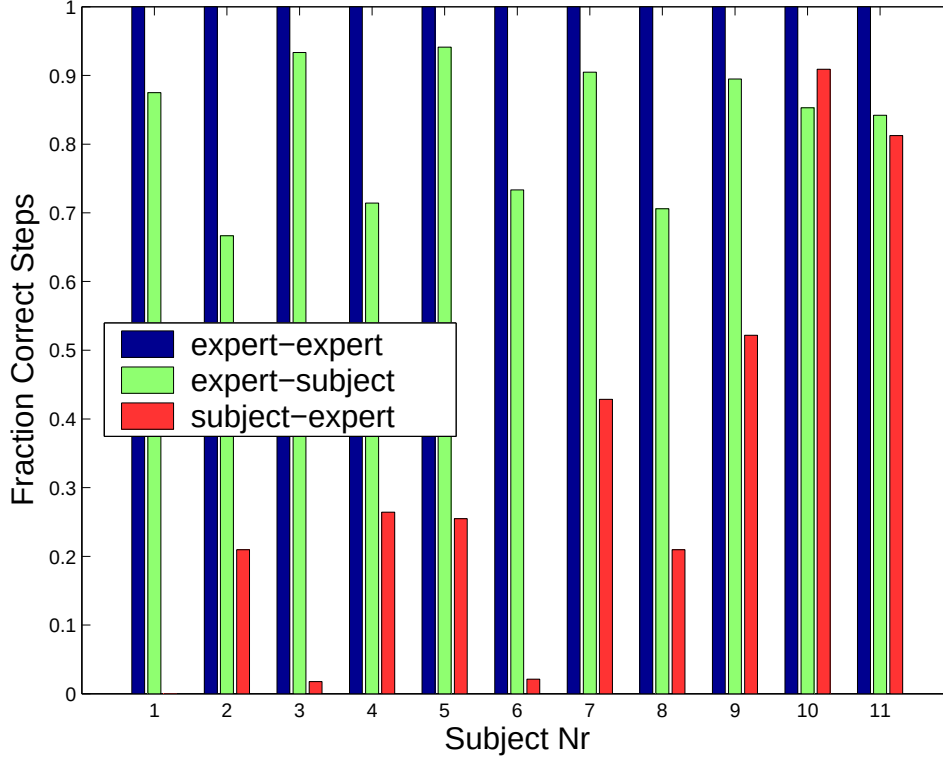


Figure 12: Error rates of SimIOHMMs trained with the expert data and tested on the expert data (expert-expert), trained on the subject data and tested on the expert data (subject-expert), and trained on the expert data and tested on the subject data (expert-subject).

The SimIOHMM trained with the expert data predicts perfectly the expert actions. The same figure also shows how well a procedure trained using the subject data can predict the expert actions (i.e., the “correct” actions at each point of the procedure). In spite of the inherently noisy nature of the subject traces, the correct prediction rate was 82% (203 out of 246 actions).

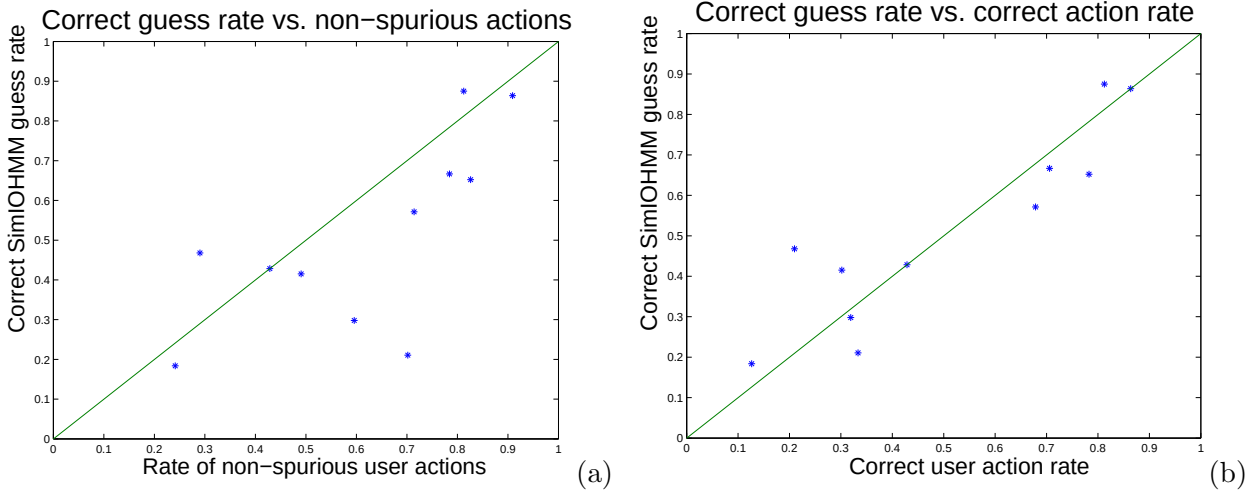


Figure 13: Results of using a SimIOHMM trained with expert data to predict the actions performed by the subjects. (a) Fraction of SimIOHMM correct guesses as a function of subject spurious action rate (fraction). (b) Rate of SimIOHMM correct predictions as a function of the subject correct guess rate.

The rightmost bar in the figure shows the correct prediction rate of an SimIOHMM trained on the expert data and tested against the subject, noisy data. Here, the fraction of correct predictions varies widely, but is strongly correlated to the fraction of non-spurious actions contained in the subject trace (spurious actions are those that do not appear in expert traces): the correlation coefficient is 0.72), as shown in Figure 13(a). Figure 13(b) shows the correct prediction (guess) rate as a function of the correct prediction rate of the subject (we counted as mistakes spurious, out-of-order, and off-track

actions) here, the correlation coefficient is 0.90.

4.4 Discussion

From the data collected in the user study we conclude that the subjects had trouble following the written instructions, which, although somewhat poorly formatted and with somewhat misleading images, were nevertheless correct, complete, and could be executed by the experts.

We observed that users had trouble when they encountered decision points in the procedure, failed to match instructions to widgets on the screen, and lost track of where they were in the procedure. Additionally, subjects were confused when the figures in the written procedure were not immediately adjacent to the corresponding text. As a consequence, users made wrong decisions, skipped steps, performed the wrong steps, and had to backtrack and undo their changes. This is reflected in the difference between the number of actions performed by the expert and by the subject as depicted in Figure 11. Additionally, four of the eleven subjects failed to produce the required machine configuration.

Since the instruction document was obtained from a self-help internal website, and its quality was comparable to that of the other documents on the site, we conclude that the difficulties experienced by the subjects reflect the difficulties that real users experience when following self-help directions. Testing an SimIOHMM trained on expert data on the subject traces provides

us with another qualitative measure of the difficulty that the subjects had in following the written instruction. The SimIOHMM tries to predict the action that an expert would have performed when facing the same situation as each subject. Some subjects, for example 3 and 6, performed very few actions that matched the ones predicted by the expert-trained SimIOHMM. In the case of subject 1, every single action was unpredictable by the expert-trained SimIOHMM—recall that subject 1 performed 66 actions that were not present in any of the 11 expert traces. Conversely, users 10 and 11 performed very few “spurious” actions (2 out of 22 and 3 out of 16 respectively), and otherwise followed the written instructions quite literally; here the Sheepdog learning algorithm correctly predicted the non-idiosyncratic actions.

The study also hints at the performance of the off-track tracking mechanism. The experiments simulate a guidance scenario where the SimIOHMM is asked to suggest an action and the user is allowed to deviate from the suggestions. After each prediction, the SimIOHMM is notified of the actual action performed by the user. If the user action differs from the prediction, the SimIOHMM enters the off-track mode and attempts to identify whether the user has jumped to a different part of the procedure. Figure 13(b) suggests that the tracking algorithm can quickly recover when the user goes back on track after making mistakes. If that were not the case, the figure would contain points substantially below the 45 degrees line.

Figure 12 shows that the Sheepdog learning algorithm induced a procedure structure that was consistent with the expert traces. In fact, the resulting executable procedure could be used to correctly predict all the traces produced by the experts. Hence, when trained with expert traces, Sheepdog can perform the required task as well as the expert. Hence, a procedure trained with traces devoid of errors can effectively assist a user in performing a task.

Even when trained with very noisy data, that is, with the subject traces, Sheepdog could predict the appropriate actions more than 82% of the time. To appreciate the amount of noise in the data, recall that only 56% of the subject actions were correct, while the rest were either spurious or were performed out of sequence. We believe that the predictive ability of the model trained with the subject data shows the ability of SimIOHMM to capture well-worn paths. In spite of the many mistakes and deviations from the prescribed procedure, taken as a group the subjects performed more often correct actions than incorrect actions, and SimIOHMM successfully captured them. This result is very positive: it shows that parallel collaborative PBD is a viable approach to collecting procedural knowledge, and is robust to the unavoidable presence of mistakes in the examples.

5 Related Work

For a broad overview of the Programming-By-Demonstration field, we direct the interested reader to the classical references [1, 2].

Existing PBD systems use learning algorithms that differ along three main dimensions: (1) the data used for induction, (2) the complexity of the induced model, and (3) the role of the user in the induction process.

1. Many existing PBD systems use only information about user actions to capture the target procedure. For example, Familiar [13] observes user actions in the Macintosh Finder application, and recognizes repetitive sequences of such actions. Using this approach, the learning algorithm cannot infer constructs, such as decision points, that depend on the content of the UI. A few systems, like SMARTedit [14], instead capture states of an application GUI, and build a procedure model by identifying the actions that are consistent with the observed input changes. Using this approach, the learning algorithm implicitly assumes that it is always possible to identify the action that produces a specific change to the GUI, and also needs to rely on a very precise model of the target application. In fact, the learning algorithm must rely on knowledge of the specific *release* of the target application: in order to infer the procedure model, the learner must know exactly what actions are required to yield the desired change in GUI content. Some PBD

systems overcome the limitations of using just snapshots or actions by accounting for both in the induction process.

2. Most existing programming-by-demonstration systems, such as [15] and Familiar [16] have focused on simple, repetitive tasks, where the user iterates over the same sequence of actions multiple times. Fixed-length loop induction, where loops are not constrained to contain all the actions in the procedure, was a feature of Eager [17], but this system could not detect any structure within a loop beyond a sequence of steps. The paper by Lau, Domingos, and Weld [14] contains recent advances on using version-space algebra as the learning algorithm, as well discussion of previous work. In this paper, the authors describe a system, SMARTpython, that learns a grammar containing complex structures such as loops and conditionals: this grammar can generate deterministic models having very complex structure. The literature contains numerous techniques for modeling sequences of statistically dependent input-output pairs that could be used in PBD. In particular, we cite Input-Output Hidden Markov Models (IOHMMs) [3] and Conditional Random Fields [18]. The SimIOHMM used by Sheepdog is an extension of the IOHMMs. Unlike the other approaches described above, these statistical techniques produce probabilistic models, and therefore can capture non-deterministic aspects of procedures, such as the existence of alternative paths through a procedure and the prefer-

ences of the user population regarding which path to select. Although the statistical methods are clearly more powerful than the other techniques described above, it is very difficult to transform the model they produce into a human-readable description of the procedure.

3. PBD systems differ on the role played by the user in the induction process. Many of the cited systems, such as Familiar, SMARTedit, and SMARTpython require the user to solve the alignment problem by stating, more or less explicitly, the step associated with each action. For example, when the system induces simple structures, such as the outer loop containing a fixed number of steps produced by SMARTedit, the user provides the alignment by explicitly stating the beginning and end of an iteration. SMARTpython induces programs having a complex structure provided that the user states the program identifier (the label) of the step being executed at each point in time. In other systems, such as Gamut [19], the alignment problem is actually solved by requiring the user to direct the system every time the induction algorithm fails to correctly predict an action. In SMARTedit the user also can inspect the different hypotheses being considered in the step generalization process and select one. The Sheepdog system requires the user to press a start button to begin recording, and a stop button to denote the end of the procedure. However, since the procedure structure is arbitrarily complex, the user is actually not solving the

alignment problem for the system.

Sheepdog has features that, to our knowledge, do not appear in previous PBD systems. The first is an authoring facility that allows the user to edit the description of the procedure that is presented to the user, without affecting the underlying automatically induced task representation. The second is the ability to track a user who diverges from the learned procedure model and to regain the initiative when the user is back on track.

6 Conclusions and Future Work

This paper introduced the field of parallel collaborative PBD as a procedural know-how capture mechanism. A parallel collaborative PBD system independently records multiple experts who perform a specific task and combines the recording into an executable procedure model, via a learning algorithm. We have analyzed the different components of a PBD systems—instrumentation, abstraction, learning, and playback—and identified where traditional PBD (focused on end-user-programming) fails to provide the methods and techniques to support parallel collaborative PBD. We have suggested solutions to these shortcomings and described their implementation in an actual system called Sheepdog.

The algorithms and methods proposed in this paper have general applicability. The architecture implementing them is highly modular and can

be used to support collaborative PBD on a variety of platforms: operating systems such as Windows and Linux, web-browsers, and individual applications, by appropriately modifying the instrumentation and the abstraction layer. The initial experiments conducted using Sheepdog on a variety of learning tasks are very promising. They suggest that PBD can correctly learn procedures that regular users often fail to perform correctly, even when following written instructions.

The Sheepdog experience is the only the first step in the exploration of collaborative PBD. It is well suited for environments with numerous experts, where the data collection should be done unobtrusively, and where the end user desires to complete the task rather than to learn how to perform it. The main features of Sheepdog that are well-suited for this environment are: the recording layer, which could operate with a simple user interface; the learning algorithm, which combines independently recorded traces into a probabilistic model of the task, which outlines the well-worn paths through the procedure; the authoring UI, which allows an author to create a human-readable description of the task containing only the details deemed necessary for the intended end-users; the playback engine, which allows the end-user to take over and deviate from the prescribed path; and the widget finding algorithm, which deals with differences across user environment configurations.

The space of collaborative PBD includes scenarios where the Sheepdog

learning algorithm is not ideal: when the number of experts from whom data is collected is small, or where the author wants immediate, real-time feedback on how the learning algorithm is interpreting the observations, SimIOHMM is not a viable choice and different algorithms should be used. For example, when both incremental learning and the ability to directly edit the procedure model are desired, Augmentation-Based Learning [9] would be the algorithm of choice. When different authors are allowed to incrementally modify a procedure model in parallel, Distributed Augmentation-Based Learning [20] provides an additional reconciliation facility.

Similarly, when the goal of the PBD system is shifted from automatic or collaborative execution towards intelligent documentation or tutorials, a different user experience, such as the one provided by the DocWizards system [21] might be preferable to that offered by Sheepdog.

Nevertheless, the vast majority of the lessons learned while developing Sheepdog carry over to these new scenarios and form a foundation for general distributed PBD.

Acknowledgements

We would like to thank Peter Franaszek, for the insightful discussions that lead to the formulation of this project, and John Turek, for his support and for providing useful suggestions on the directions in which to steer our investigation.

References

- [1] A. Cypher, ed., *Watch what I do: Programming by demonstration*. Cambridge, MA: MIT Press, 1993.
- [2] H. Lieberman, ed., *Your Wish is My Command: Programming By Example*. Morgan Kaufmann, 2001.
- [3] Y. Bengio and P. Frasconi, “Input-Output HMM’s for sequence processing,” *IEEE Trans. Neural Networks*, vol. 7, pp. 1231–1249, Sept. 1996.
- [4] R. Fikes, P. Hart, and N. Nilsson, “Learning and executing generalized robot plans,” *Artificial Intelligence*, vol. 3, pp. 251–288, 1973.
- [5] P. Frasconi and Y. Bengio, “An EM approach to grammatical inference: Input/Output HMMs,” in *Proc. IEEE Int. Conf. Pattern Recognition, ICPR ’94*, (Jerusalem), pp. 289–294, October 9-13 1994.
- [6] D. Oblinger, V. Castelli, T. Lau, and L. D. Bergman, “Similarity-based alignment and generalization,” in *Proc. Sixteenth Europ. Conf. on Machine Learning*, (Berlin), pp. 657–664, Springer, October 2005.
- [7] I. Witten and E. Frank, *Data Mining: Practical Machine Learning Tools and Techniques with Java Implementations*. Morgan Kaufmann, 1999.
- [8] L. Devroye, L. Györfi, and G. Lugosi, *A probabilistic theory of pattern recognition*. New York: Springer, 1996.

- [9] D. Oblinger, V. Castelli, and L. D. Bergman, “Augmentation-based learning, combining observations and user edits for programming-by-demonstration,” in *Proc. 2006 Int. Conf. on Intelligent User Interfaces*, (Sidney, Australia), pp. 202–209, February 2006.
- [10] A. Viterbi, “Error bounds for convolutional codes and an asymptotically optimum decoding algorithm,” *IEEE Trans. Information Theory*, vol. 13, pp. 260–269, Apr. 1967.
- [11] T. Lau, L. D. Bergman, V. Castelli, and D. Oblinger, “Sheepdog: Learning procedures for technical support,” in *Proc. 2004 Int. Conf. on Intelligent User Interfaces*, (Madeira, Portugal), pp. 106–116, 2004.
- [12] A. Guyon and D. Hinkley, *Bootstrap methods and their applications*. Cambridge University Press, 1997.
- [13] G. Paynter and W. I.H., “Automating iteration with programming by demonstration: Learning the user’s task,” in *Proc. IJCAI Workshop on Learning About Users*, (Stockholm, Sweden), Morgan Kaufmann, 1999.
- [14] T. Lau, P. Domingos, and D. S. Weld, “Learning programs from traces using version space algebra,” in *Proc. 2nd Int. Conf. on Knowledge Capture*, 2003.
- [15] T. Lau, S. A. Wolfman, P. Domingos, and W. D. S., “Programming by demonstration using version space algebra,” *Mach. Learn.*, vol. 53, no. 1-2, pp. 111–156, 2003.

- [16] G. Paynter, *Automating iterative tasks with programming by demonstration*. Ph.D. dissertation, The University of Waikato, New Zealand, 2000.
- [17] A. Cypher, “Eager: Programming repetitive tasks by demonstration,” in *Watch What I Do: Programming by Demonstration* (A. Cypher, ed.), pp. 205–217, Cambridge, MA: MIT Press, 1993.
- [18] J. Lafferty, A. McCallum, and F. Pereira, “Conditional random field: Probabilistic models for segmenting and labeling sequence data,” in *Proc. Eighteenth Int. Conf. on Machine Learning*, pp. 282–289, 2001.
- [19] R. McDaniel and B. Myers, “Building applications using only demonstration,” in *Proc. 1998 Int. Conf. on Intelligent User Interfaces*, pp. 282–289, 1998.
- [20] V. Castelli and L. D. Bergman, “Distributed augmentation-based learning: A learning algorithm for distributed collaborative programming-by-demonstration,” in to appear in *Proc. 2007 Int. Conf. on Intelligent User Interfaces*, 2007.
- [21] L. D. Bergman, V. Castelli, T. Lau, and D. Oblinger, “DocWizards: a system for authoring follow-me documentation wizards,” in *Proc. 18th annual ACM Symp. on User Interface Software and Technology, UIST2005*, pp. 191–200, 2005.

Subject	On Track	Off Path	Spurious	Skipped	Total	Expert	Success
1	15	13	0	1	28	17	Y
2	41	16	5	0	62	34	N
3	21	1	1	0	23	21	Y
4	19	28	10	0	57	18	Y
5	43	4	4	0	51	38	N
6	16	0	0	0	16	15	Y
7	18	59	10	0	87	17	Y
8	22	5	1	0	28	19	Y
9	15	31	1	1	47	16	N
10	21	0	1	0	22	21	Y
11	36	7	10	0	53	30	N

Table 2: Counts of user actions: the table reports the total number of correct actions; of off-track actions (i.e., actions not described in the documentation and that clearly indicate that the user are not following the directions correctly); of spurious actions (i.e., actions not described in the documentation that are irrelevant to the task, but do not indicate that the user is not following the directions correctly); of skipped steps; and the total number of observed actions. The seventh column is the number of actions performed by the expert for the same task, while the last column denotes whether the subject correctly completed the task.